Blog App

Documentation Projet CAKE PHP

Table des matières

Création du projet avec Composer / NetBeans 3

Migrations de la table article 4

1ERE PARTIE – CRÉATION D'ARTICLES 5

– Création modèle pour articles 5

– Création template pour la vue Index.php de Articles 6

– Action détail & vue détail 6

– Action add + vue dans Articles 7

– Boutons action d'articles (création et liste) 7

- Validation des données – Ajout d'un article 8

– Action edit + vue edit dans Articles 9

– Action delete + vue dans index de Articles 10

2EME PARTIE – CREATIONS DES UTILISATEURS 12

Migrations de la table users 12

Création Modèle + Controller Users 14

Création du template Users 14

– Action index dans le UsersController 14

– Vue Index de Users 15

– Nouvelle version de UsersTable pour les validateurs 16

– Plugin de composer : hachage des mots de passes 16

– Composer update 17

– Réinstallation du plugin & Mise à jour de CakePHP 17

Affichage des créateurs d'articles + le nombre d'articles 18

– Utilisateur associé à un article 18

– Nombre d'articles créés par un utilisateur 19

– Choisir un user lors de la modification d'un article 20

Action edit dans ArticlesController 20

Ajout d'un sélecteur dans la vue edit 20

Action add dans ArticlesController 20

Ajout du sélecteur dans la vue add 20

3EME PARTIE — Fonctionnalités : les commentaires 21

Diagramme de classe Comment 21

Migration 21

Création de la table en CMD 21

Ajout de la clé étrangère dans la migration 22

– Validateur et liaison belongsTo et hasMany 23

– Méthode add dans CommentsController 24

– Création de la vue comments 24
- Les commentaires dans ArticlesController 25
- Méthode delete dans CommentsController 26
- Script d'affichage (jquery371min) 26
- Affichage du formulaire 27
- Affichage de la liste des utilisateurs 28
– Boutons remplacés par des icônes 31
- Onglets dans la navbar 32
4E PARTIE – SÉCURISATION ET CONNEXION 33
➢ Procédure CakePHP 3.0 33
➢ Action login 35
➢ Vue login – UsersController 36
- Ancien version – Cake 3.0 36
- Nouvelle version 36
➢ Utilisateur attribué à un article (auto) 39
Association des utilisateurs aux commentaires 40
5EME PARTIE – CREATION DES TAGS 44
➢ Ajout des tags sur les articles 44
➢ Action index – TagsController 45
➢ Action view – TagsController 45
➢ Action add – TagsController 46
➢ Action edit – TagsController 46
➢ Action delete – TagsController 46
➢ Relations et Validations – TagsTable 46
➢ Vue Tags – Template Tags 48
- Action Add 48
- Action edit 49
- Action index 50
- Action view 51
➢ Ajout d'une table entre Articles et Tags 53
Migrations 53
➢ Relation & Validateurs - ArticleTagsTable 54
➢ ArticleTagsController 55
- Action index 55
- Action view 55
- Action add 55
- Action edit 56

- Action delete 56
- Action deleteAT 56
➢ Rework de ArticlesController 58
- Action index : 58
- Action édit : 58
- Action détail : 59
- Action add 60
➢ Comptage 62
- Nombre d'articles sur les tags 62
- Nombre tags sur les articles 62

Diagramme de classe final 64

RECAP' DES ÉTAPES DE BASE 64

Création du projet avec Composer / NetBeans

– **Création du projet avec Composer.**
Aller dans le disque C :\ wamp64\www\ et écrire dans la barre de l'explorer « cmd »

– **Dans le CMD :**
« composer create-project --prefer-dist cakephp/app:5.* blog_app. »

– **Importation dans NetBeans**

– **Pour créer la base de données en ligne de commande avec MySQL :**

C:\wamp64\bin\mysql\mysql9.1.0\bin>mysql -u root puis :

– **Créer un utilisateur dédié avec tous les droits pour blog_app uniquement**
  ➢ Create user blog_app identified by « BlogApp011025/* »

```
mysql> create user blog_app identified by "BlogApp011025/*" ;
Query OK, 0 rows affected (0.12 sec)

mysql> create database blog_app_bdd ;
Query OK, 1 row affected (0.26 sec)

mysql>
```

```
mysql> grant all privileges on blog_app_bdd.* to blog_app ;
Query OK, 0 rows affected (0.01 sec)
```

– **Configurer BDD dans l'app blog_app**

Dans config > app_local.php

```php
    'Datasources' => [
        'default' => [
            'host' => 'localhost',
            /*
             * CakePHP will use the default DB port based on the driver selected
             * MySQL on MAMP uses port 8889, MAMP users will want to uncomment
             * the following line and set the port accordingly
             */
            //'port' => 'non_standard_port_number',

            'username' => 'blog_app',
            'password' => 'BlogApp011025/*',

            'database' => 'blog_app_bdd',
            /*
             * If not using the default 'public' schema with the PostgreSQL driver
             * set it here.
             */
            //'schema' => 'myapp',

            /*
             * You can use a DSN string to set the entire configuration
             */
            'url' => env('DATABASE_URL', null),
        ],
```

- Changer le nom de la base de données, l'username et password de l'utilisateur administrateur.

Migrations de la table article

- **Création migration pour la table articles (InitialMigration)**

```
C:\wamp64\www\blog_app\bin>cake migrations create InitialMigration

Creating file C:\wamp64\www\blog_app\config\Migrations\20251001044614_InitialMigration.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251001044614_InitialMigration.php`
```

- **Migration pour modifier la table articles (AlterArticles)**

```
C:\wamp64\www\blog_app\bin>cake migrations create AlterArticles

Creating file C:\wamp64\www\blog_app\config\Migrations\20251001055302_AlterArticles.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251001055302_AlterArticles.php`


C:\wamp64\www\blog_app\bin>cake migrations status
+--------+----------------+------------------+
| Status | Migration ID   | Migration Name   |
+--------+----------------+------------------+
| down   | 20251001044614 | InitialMigration |
| down   | 20251001055302 | AlterArticles    |
+--------+----------------+------------------+
```

– **Migration pour insérer des données dans articles**

```
C:\wamp64\www\blog_app\bin>cake migrations create InsertArticles

Creating file C:\wamp64\www\blog_app\config\Migrations\20251001060930_InsertArticles.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251001060930_InsertArticles.php`


C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251001060930 InsertArticles: migrating
 == 20251001060930 InsertArticles: migrated 0.0766s
All Done. Took 0.1397s
```

➢ Insertion des données d'articles dans le fichier migration InsertArticles (pour tester)

```php
 */
public function up(): void
{
    $articles = $this->table('articles');
    $rows = [
      ['title' => 'titre1', 'content' => 'sdsdsd', 'created' => date('Y-m-d H:i:s'), 'modified' => date('Y-m-d H:i:s')],
      ['title' => 'titre2', 'content' => 'content2', 'created' => date('Y-m-d H:i:s'), 'modified' => date('Y-m-d H:i:s')],
      ['title' => 'titre3', 'content' => 'content3', 'created' => date('Y-m-d H:i:s'), 'modified' => date('Y-m-d H:i:s')],
      ['title' => 'titre4', 'content' => 'content4', 'created' => date('Y-m-d H:i:s'), 'modified' => date('Y-m-d H:i:s')],
      ['title' => 'titre5', 'content' => 'content5', 'created' => date('Y-m-d H:i:s'), 'modified' => date('Y-m-d H:i:s')]
    ];
    $articles->insert($rows)->saveData();
}

public function down(): void
{
    $this->execute('DELETE from articles');
}
```

1ERE PARTIE – CRÉATION D'ARTICLES

– Création modèle pour articles

➢ Création de fichier ArticlesTable.php

```php
class ArticlesTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
                ->notEmptyString('title', __('Veuillez renseigner un titre'))
                ->notEmptyString('content', _('Veuillez renseigner une description'));

        return $validator;
    }
}
```

- Ajout des validateurs title et content pour les définir comme des champs obligatoires à remplir.

– Création template pour la vue Index.php de Articles

```php
<h1>Tous les articles du Blog</h1>

<?=
$this->html->link('Créer un article', ['action' => 'add'], ['class' => 'button', 'style' => 'background-color : #266EE5;']);
?>

<table>
    <tr>
        <th>Id</th>
        <th>Titre</th>
        <th>Date de création</th>
        <th>Date de modification</th>
        <th> Action </th>
    </tr>

    <!-- Ici se trouve l'itération sur l'objet query de notre $mesArticles, l'affichage des infos des articles -->
    <?php foreach ($mesArticles as $article): ?>
        <tr>
            <td><?= $article->id ?></td>
            <td>
                <?=
                $this->html->link($article->title, [
                    'controller' => 'articles',
                    'action' => 'detail',
                    $article->id]);
                //l'url généré sera de la forme /articles/detail/…
                ?>
            </td>

            <td><?= $article->created->format(DATE_RFC850) ?></td>
            <td><?= $article->modified->format(DATE_RFC850) ?></td>
```

– Action détail & vue détail

```php
    public function detail($id = null) {
        try {
            $leArticle = $this->Articles->get($id);
        } catch (\Exception $ex) {
            if ($id == null) {
                $this->Flash->error(__("L'action detail doit être appelé avec un identifiant"));
            } else {
                $this->Flash->error(__("L'article {0} n'existe pas", $id));
            }
            return $this->redirect(['action' => 'index']);
        }

        $this->set(compact('leArticle'));
    }
```

```php
<h1><?= h($leArticle->title) ?></h1>
<p><?= nl2br(h($leArticle->content)) ?></p>
<p>
    <small>Created : <?= $leArticle->created->format(DATE_RFC850) ?></small>
</p>
<br></br>

<?= $this->html->link("Modifier l'article", ['controller' => 'articles', 'action' => 'edit', $leArticle->id], ['class' => 'button']); ?>

<?=
$this->html->link('Retour à la liste des articles', ['action' => 'index'], ['class' => 'button']);
?>
```

– Action add + vue dans Articles

```php
public function add() {
    $leNewArticle = $this->Articles->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewArticle = $this->Articles->patchEntity($leNewArticle, $this->request->getData());
        if ($this->Articles->save($leNewArticle)) {
            $this->Flash->success(__("L'article a été sauvegardé."));
            return $this->redirect(['action' => 'index']);
        } else {
            $this->Flash->error(__("Impossible d'ajouter votre article."));
        }
    }
    $this->set(compact('leNewArticle'));
}
```

```php
<h1>Ajouter un article</h1>
<?php
    echo $this->Form->create($leNewArticle);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button(__("Sauvegarder l'article"));
    echo $this->Form->end();
    echo $this->html->link('Retour à la liste des articles', ['action' => 'index'],['class' => 'button']);
```

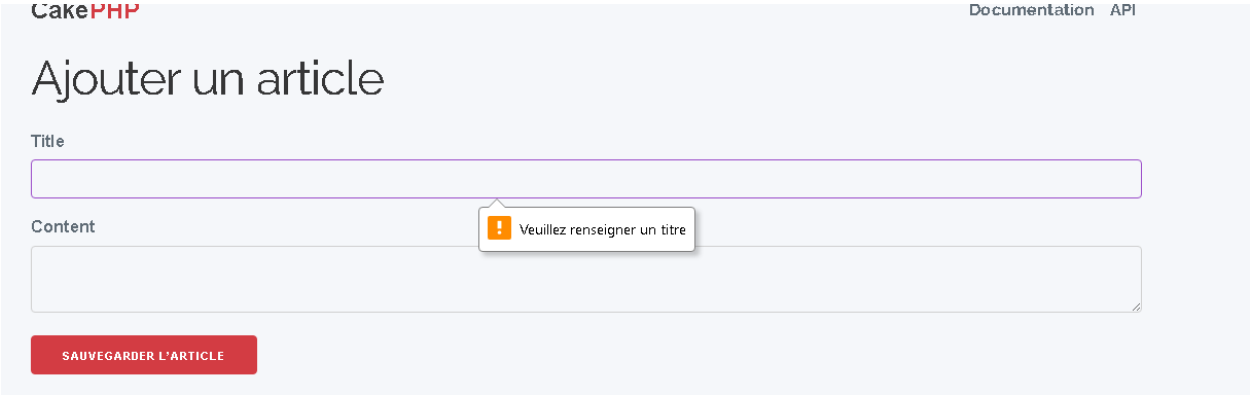
– Boutons action d'articles (création et liste)

- Création d'un bouton pour créer les articles
- Un bouton pour revenir sur la liste des articles

Le résultat (*code dans les titres précédents*) :

- Validation des données – Ajout d'un article

➢ Valider les données avec une alerte en cas de champs vide.



Ajouter des validateurs dans **CommentsTable**

```
public function validationDefault(Validator $validator): Validator {
    $validator
            ->notEmptyString('title', __('Veuillez renseigner un titre'))
            ->notEmptyString('content', __('Veuillez renseigner une description'));

    return $validator;
}
```

– Action edit + vue edit dans Articles

```php
public function edit($id = null) {
    try {
        $leArticle = $this->Articles->get($id);
    } catch (\Exception $ex) {
        if ($id == null) {
            $this->Flash->error(__("L'action edit doit être appelé avec un identifiant"));
        } else {
            $this->Flash->error(__("L'article {0} n'existe pas", $id));
        }
        return $this->redirect(['action' => 'index']);
    }

    if ($this->request->is(['post', 'put'])) {
        $this->Articles->patchEntity($leArticle, $this->request->getData());
        if ($this->Articles->save($leArticle)) {
            $this->Flash->success(__('Votre article a été mis à jour.'));
            return $this->redirect(['action' => 'index']);
        } else {
            $this->Flash->error(__('Impossible de mettre à jour votre article.'));
        }
    }

    $this->set(compact('leArticle'));
}
```

Fonction edit dans ArticlesController.php

```php
<h1>Modifier l'article "<?= $leArticle->title?>" (id = <?= $leArticle->id ?>)</h1>
<?php
    echo $this->Form->create($leArticle);
    echo $this->Form->control('title');
    echo $this->Form->control('content', ['rows' => '3']);
    echo $this->Form->button(__("Mettre à jour l'article"));
    echo $this->Form->end();
?>
<br/>

<?=
$this->html->link('Retour aux articles',
        ['controller' => 'articles', 'action' => 'index'],
        ['class' => 'button']);
?>
```

Vue edit dans template > Articles > edit.php

Le résultat :

– Action delete + vue dans index de Articles

Une vue de delete n'est pas nécéssaire. On fait directement dans l'index de Articles :

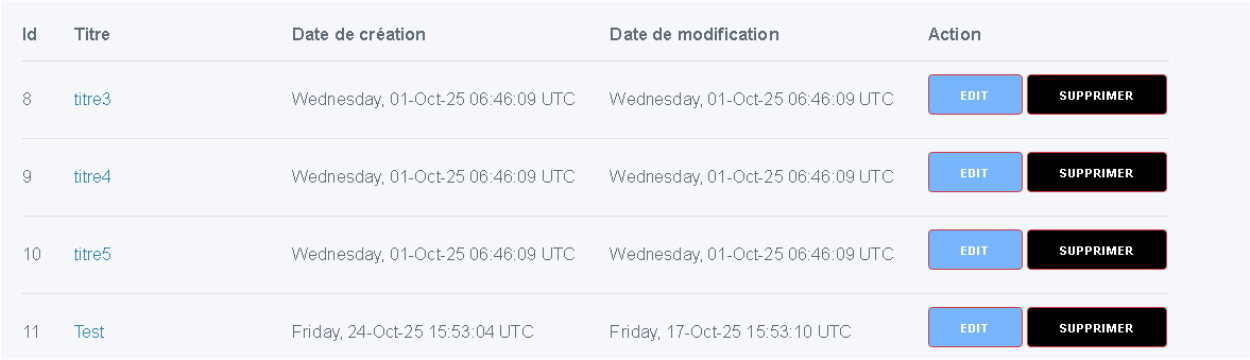
Fonction delete dans ArticlesController.php

```
public function delete($id = null) {
    $this->request->allowMethod(['post', 'delete']);
    $leArticle = $this->Articles->get($id);
    if ($this->Articles->delete($leArticle)) {
        $this->Flash->success(__("L'article {0} d' id {1} a bien été supprimé ! ", $leArticle->title, $leArticle->id));
    return $this->redirect(['action' => 'index']);
    }
}
```

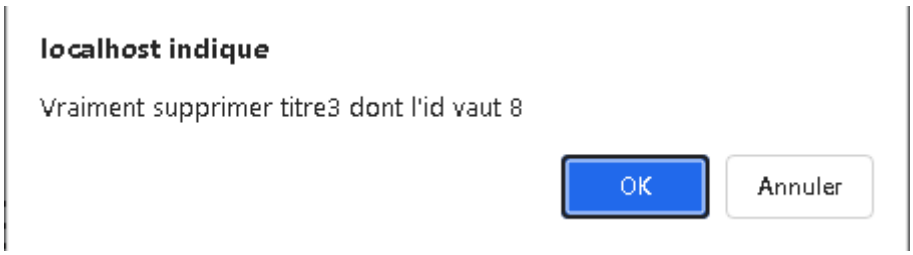
Affichage dans index :

```
<td> <?= $this->html->link("Edit", ['controller' => 'articles', 'action' => 'edit', $article->id],
        ['class' => 'button', 'style' => 'background-color : #77b5fe;']); ?>
<?=
    $this->Form->postLink(
            __('Supprimer'),
            ['action' => 'delete', $article->id],
            ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $article->title, $article->id),
                'class' => 'button',
                'style' => 'background-color : Black']);
    ?>
</td>
```

Le résultat :



Id	Titre	Date de création	Date de modification	Action
8	titre3	Wednesday, 01-Oct-25 06:46:09 UTC	Wednesday, 01-Oct-25 06:46:09 UTC	EDIT SUPPRIMER
9	titre4	Wednesday, 01-Oct-25 06:46:09 UTC	Wednesday, 01-Oct-25 06:46:09 UTC	EDIT SUPPRIMER
10	titre5	Wednesday, 01-Oct-25 06:46:09 UTC	Wednesday, 01-Oct-25 06:46:09 UTC	EDIT SUPPRIMER
11	Test	Friday, 24-Oct-25 15:53:04 UTC	Friday, 17-Oct-25 15:53:10 UTC	EDIT SUPPRIMER



Confirmation (venant du __confirm) pour supprimer l’articles

2EME PARTIE – CREATIONS DES UTILISATEURS

Migrations de la table users

- Création de la migration pour la table users

```
C:\wamp64\www\blog_app\bin>cake bake migration CreateUsers name:string lastname:string age:integer email:string username
:string password:string created modified

Creating file C:\wamp64\www\blog_app\config\Migrations\20251008064549_CreateUsers.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251008064549_CreateUsers.php`


C:\wamp64\www\blog_app\bin>cake migrations status
+--------+----------------+------------------+
| Status | Migration ID   | Migration Name   |
+--------+----------------+------------------+
| up     | 20251001044614 | InitialMigration |
| up     | 20251001055302 | AlterArticles    |
| up     | 20251001060930 | InsertArticles   |
| down   | 20251008064549 | CreateUsers      |
+--------+----------------+------------------+
```

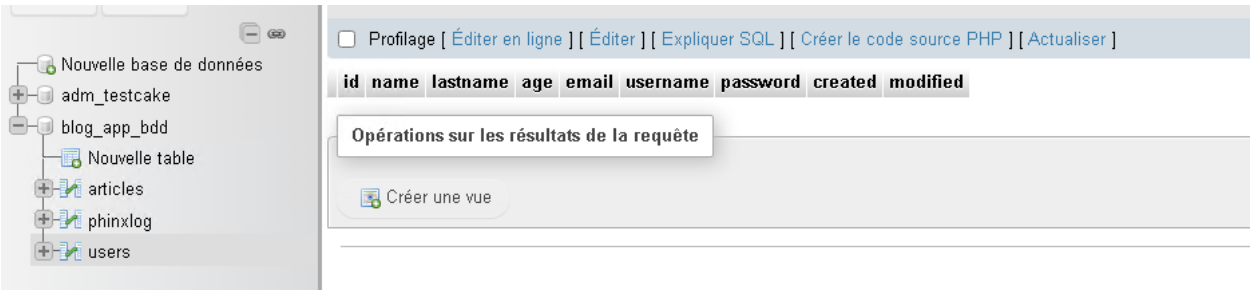
➔ « cake bake migration CreateUsers .. » création des colonnes.
➔ **Cake migrations status** : verification du status de la migration.

```
C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251008064549 CreateUsers: migrating
 == 20251008064549 CreateUsers: migrated 0.6106s
All Done. Took 0.6610s
```

➔ Commande migration pour migrer.

```
C:\wamp64\www\blog_app\bin>cake migrations create InsertUsers

Creating file C:\wamp64\www\blog_app\config\Migrations\20251008065156_InsertUsers.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251008065156_InsertUsers.php`
```

➔ Création de la migration InsertUsers.



➢ Vérification dans phpMyAdmin la création des colonnes.

- **Insertion des 5 utilisateurs**

Initial Migration (qui sert à la création directe de la table) :

```
...

$users = $this->table('users');
$users->create();
```

Dans InsertUsers.php :

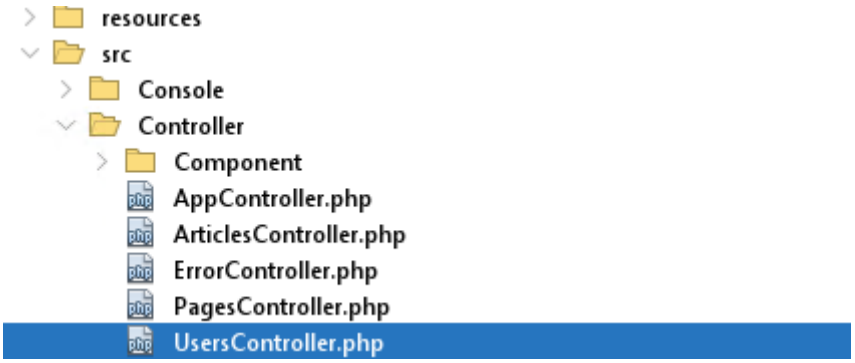
```
*/
public function up(): void
{
    $users = $this->table('users');
    $rows = [
      ['name' => 'TestPrenom1', 'lastname' => 'TestNom1', 'age' => '18', 'email' => 'test@gmail.com', 'username' => 'user1', 'password' => 'mo
      ['name' => 'TestPrenom2', 'lastname' => 'TestNom2', 'age' => '19', 'email' => 'test@gmail.com', 'username' => 'user2', 'password' => 'mo
      ['name' => 'TestPrenom3', 'lastname' => 'TestNom3', 'age' => '8', 'email' => 'test@gmail.com', 'username' => 'user3', 'password' => 'mot
      ['name' => 'TestPrenom4', 'lastname' => 'TestNom4', 'age' => '20', 'email' => 'test@gmail.com', 'username' => 'user4', 'password' => 'mo
      ['name' => 'TestPrenom5', 'lastname' => 'TestNom5', 'age' => '28', 'email' => 'test@gmail.com', 'username' => 'user5', 'password' => 'mo
    ];
    $users->insert($rows)->saveData();
}
```

➔ Rollback de la migration pour supprimer et migrate pour réinsérer les données.

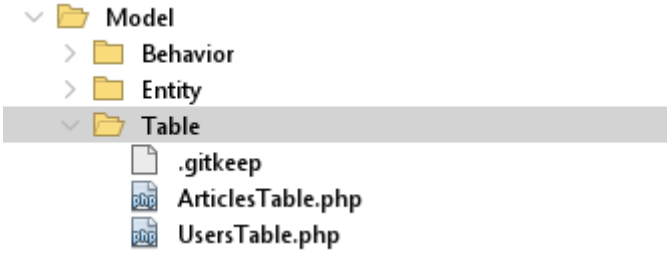
```
C:\wamp64\www\blog_app\bin>cake migrations rollback

 == 20251001060930 InsertArticles: reverting
 == 20251001060930 InsertArticles: reverted 0.0231s
All Done. Took 0.1258s



C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251001060930 InsertArticles: migrating
 == 20251001060930 InsertArticles: migrated 0.0177s

 == 20251008064549 CreateUsers: migrating
 == 20251008064549 CreateUsers: migrated 0.0665s

 == 20251008065156 InsertUsers: migrating
 == 20251008065156 InsertUsers: migrated 0.0100s
All Done. Took 0.1448s
```

Création Modèle + Controller Users

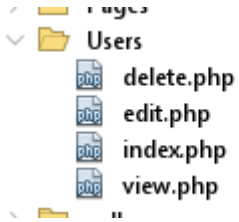
➔ Création du controller UsersController.php



➔ Création la table UsersTable.php



Création du template Users



➔ Création de toutes les pages d'actions associés à Users dans template (*index, view, edit, delete*)

– Action index dans le UsersController

```php
<?php

namespace App\Controller;

class UsersController extends AppController {

    public function index() {
        //on récupère tous les utilisateurs et on les stocke dans $mesUsers
        $mesUsers = $this->Users->find()->all();
        $this->set(compact('mesUsers')); //envoie à la vue le contenu de $mesUsers dans $rep qui sera utiliseable
    }
}
```

UsersTable **pour le moment** :

```php
<?php

namespace App\Model\Table;

use Cake\ORM\Table;
use Cake\Validation\Validator;

class UsersTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
                ->notEmptyString('title', __('Veuillez renseigner un titre'))
                ->notEmptyString('content', _('Veuillez renseigner une description'));

        return $validator;
    }
}
```

– Vue Index de Users

```php
<h1>Tous les utilisateurs du Blog</h1>

<table>
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Last Name</th>
        <th>Age</th>
        <th>Email</th>
        <th>Username</th>
        <th>Password</th>
        <th>Date de création</th>
        <th>Date de modification</th>
        <th>Action</th>
    </tr>

    <?php foreach ($mesUsers as $user): ?>
        <tr>
            <td><?= $user->id ?></td>
            <td>
                <?=
                $this->html->link($user->name, [
                    'controller' => 'users',
                    'action' => 'view',
                    $user->id]);
                //l'url généré sera de la forme /articles/detail/…
                ?>
            </td>

            <td><?= $user->lastname ?></td>
            <td><?= $user->age ?></td>
            <td><?= $user->email ?></td>
            <td><?= $user->username?></td>
            <td><?= $user->password ?></td>
            <td><?= $user->created->format(DATE_RFC850) ?></td>
            <td><?= $user->modified->format(DATE_RFC850) ?></td>


            <td> <?= $this->html->link("Edit", ['controller' => 'articles', 'action' => 'edit', $user->id],
                    ['class' => 'button', 'style' => 'background-color : #266EE5;']);
                ?>
                <?=
                $this->Form->postLink(
                        __('Supprimer'),
                        ['action' => 'delete', $user->id],
                        ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $user->title, $user->id),
                            'class' => 'button',
```

Activer Windows
Accédez aux paramètres pour activer Windows

– Nouvelle version de UsersTable pour les validateurs

```php
    public function validationDefault(Validator $validator): Validator {
        $validator
                ->notEmptyString('name', __('Veuillez renseigner un nom'))
                ->notEmptyString('password', _('Veuillez renseigner une description'))
                ->notEmptyString('username', _('Veuillez renseigner un username'))
                ->notEmptyString('username', 'Username is required')
                ->add('username', 'unique', [
                    'rule' => 'validateUnique',
                    'provider' => 'table',
                    'message' => "Ce nom d'utilisateur est déjà prit."
                ])
                ->notEmptyString('password', _('Veuillez renseigner une description'))
                ->notEmptyString('age', _('Veuillez renseigner un age'))
                ->numeric('age', _('La valeur dddoit être un nombre'))
                ->notEmptyString('email', __('un email est nécéssaire'))
                ->email('email', __('Veuillez fournir une bonne adresse mail'));
        return $validator;
    }
}
```

- **Validateurs obligatoires : name, password, username, age, email.**

Le ; est à la dernière règle puisque **<u>$validator est un objet</u>**

– Plugin de composer : hachage des mots de passes

```
C:\wamp64\www\blog_app>composer require "cakephp/authentication:^3.3"
./composer.json has been updated
Running composer update cakephp/authentication
Loading composer repositories with package information
Updating dependencies
Lock file operations: 1 install, 0 updates, 0 removals
  - Locking cakephp/authentication (3.3.2)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 1 install, 0 updates, 0 removals
  - Installing cakephp/authentication (3.3.2): Extracting archive
1 package suggestions were added by new dependencies, use `composer suggest` to see details.
Generating autoload files
58 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
```

- ➔ Plugin pour le mot de passe. cakephp/authentication:^3.3

– Composer update

```
C:\wamp64\www\blog_app>composer update
Loading composer repositories with package information
Updating dependencies
Lock file operations: 0 installs, 4 updates, 0 removals
  - Upgrading cakephp/bake (3.4.0 => 3.5.0)
  - Upgrading cakephp/debug_kit (5.1.3 => 5.1.4)
  - Upgrading cakephp/migrations (4.7.2 => 4.8.0)
  - Upgrading phpunit/phpunit (12.3.15 => 12.4.1)
Writing lock file
Installing dependencies from lock file (including require-dev)
Package operations: 0 installs, 4 updates, 0 removals
  - Downloading cakephp/bake (3.5.0)
  - Downloading cakephp/debug_kit (5.1.4)
  - Downloading cakephp/migrations (4.8.0)
  - Downloading phpunit/phpunit (12.4.1)
  - Upgrading cakephp/bake (3.4.0 => 3.5.0): Extracting archive
  - Upgrading cakephp/debug_kit (5.1.3 => 5.1.4): Extracting archive
  - Upgrading cakephp/migrations (4.7.2 => 4.8.0): Extracting archive
  - Upgrading phpunit/phpunit (12.3.15 => 12.4.1): Extracting archive
Generating autoload files
58 packages you are using are looking for funding.
Use the `composer fund` command to find out more!
No security vulnerability advisories found.
```

Erreur sur le hachage du mot de passe ?

➔ Il faut mettre à jour le CakePHP et refaire l'installation du plugin pour faire en sorte que le Password Hasher puisse fonctionner.

– Réinstallation du plugin & Mise à jour de CakePHP

```
C:\wamp64\www\blog_app\bin>cake bake model Users
One moment while associations are detected.

Baking table class for Users...

File 'C:\wamp64\www\blog_app\src\Model\Table\UsersTable.php' exists
Do you want to overwrite? (y/n/a/q)
[n] > n
Skip 'C:\wamp64\www\blog_app\src\Model\Table\UsersTable.php'

Deleted 'C:\wamp64\www\blog_app\src\Model\Table\.gitkeep'

Baking entity class for User...
```

➔ « Non » pour le premier, car c'est déjà généré. Si ça ne l'est pas, salors « y »

```
File 'C:\wamp64\www\blog_app\src\Model\Entity\User.php' exists
Do you want to overwrite? (y/n/a/q)
[n] > y
Wrote 'C:\wamp64\www\blog_app\src\Model\Entity\User.php'
Deleted 'C:\wamp64\www\blog_app\src\Model\Entity\.gitkeep'

Baking test fixture for Users...

Creating file C:\wamp64\www\blog_app\tests\Fixture\UsersFixture.php
Wrote 'C:\wamp64\www\blog_app\tests\Fixture\UsersFixture.php'
Deleted 'C:\wamp64\www\blog_app\tests\Fixture\.gitkeep'
Bake is detecting possible fixtures...

Baking test case for App\Model\Table\UsersTable ...

Creating file C:\wamp64\www\blog_app\tests\TestCase\Model\Table\UsersTableTest.php
Wrote 'C:\wamp64\www\blog_app\tests\TestCase\Model\Table\UsersTableTest.php'
Done
```

➔ « Oui » pour le deuxième car le fichier **user.php** était déjà fait (pour ma part) à la main. On le recréer.

```php
    protected function _setPassword(string $password) : ?string {
        return (new DefaultPasswordHasher())->hash($password);
    }
```

➔ Dans user.php

Affichage des créateurs d'articles + le nombre d'articles

– Utilisateur associé à un article

Dans ArticlesController, fonction index() :

```php
public function index() {
    //on récupère tous les posts et on les stocke dans $mesArticles
    $mesArticles = $this->Articles->find()->contain([
    'Users' => function ($q) {
        return $q
        ->select(['username','email']);
    }])->all();

    $this->set(compact('mesArticles')); //envoie à la vue le contenu de $mesArticles dans $rep qui sera utiliseable
}
```

Dans index.php du template de Articles :

Dans <table> : <th> Créé par : </th>

Puis dans le foreach($mesArticles as $article) :

```
<td><?= $article->user->username ?></td>
```

Le résultat :

Id	Titre	Créer par :	Date de création	Date de modification	Action
20	titre2	user4	Wednesday, 08-Oct-25 07:11:31 UTC	Friday, 31-Oct-25 04:33:02 UTC	EDIT SUPPRIMER

– Nombre d'articles créés par un utilisateur

Dans UsersController, fonction index() :

```
class UsersController extends AppController {

    public function index() {
//on récupère tous les utilisateurs et on les stocke dans $mesUsers
        $mesUsers = $this->Users->find()->contain([
                    'Articles' => function ($q) {
                        return $q
                                ->select(['user_id']);
                    }])->all();

        $this->set(compact('mesUsers')); //envoie à la vue le contenu de $mesUsers dans $rep qui sera utiliseable
    }
```

Dans index.php du template Users :

Dans la balise <tr> : <th>Nb d'articles créés : </th>

Puis dans le foreach :

```
<td><?= count($user->articles) ?></td>
```

Le résultat :

Id	Name	Last Name	Age	Email	Username	Nb d'art. créés :	Date de création	Date de modification	Action
28	TestPrenom1	TestNom1	18	test@gmail.com	user1	4	Wednesday, 29-Oct-25 07:23:18 UTC	Wednesday, 29-Oct-25 07:23:18 UTC	EDIT SUPPRIMER

– Choisir un user lors de la modification d'un article

Action edit dans ArticlesController

```
public function edit($id = null) {
    $mesUsers = $this->fetchTable('Users')
            ->find('list', keyField: 'id', valueField: 'username')
            ->toArray();
```

```
$this->set(compact('leArticle', 'mesUsers'));
```

Ajout d'un sélecteur dans la vue edit

Ligne surligné (user_id)

```
echo $this->Form->control('content', ['rows' => '3']);
echo $this->Form->control('user_id',['options' => $mesUsers, 'label' => 'Selectionnez un utilisateur']);
echo $this->Form->button(__("Mettre à jour l'article"));
```

Action add dans ArticlesController

```
public function add() {
    $mesUsers = $this->fetchTable('Users')
            ->find('list', keyField: 'id', valueField: 'username')
            ->toArray();

    $leNewArticle = $this->Articles->newEmptyEntity();
    if ($this->request->is('post')) {
        $leNewArticle = $this->Articles->patchEntity($leNewArticle, $this->request->getData());
        if ($this->Articles->save($leNewArticle)) {
            $this->Flash->success(__("L'article a été sauvegardé."));
            return $this->redirect(['action' => 'index']);
        } else {
            $this->Flash->error(__("Impossible d'ajouter votre article."));
        }
    }
    $this->set(compact('leNewArticle', 'mesUsers'));
}
```

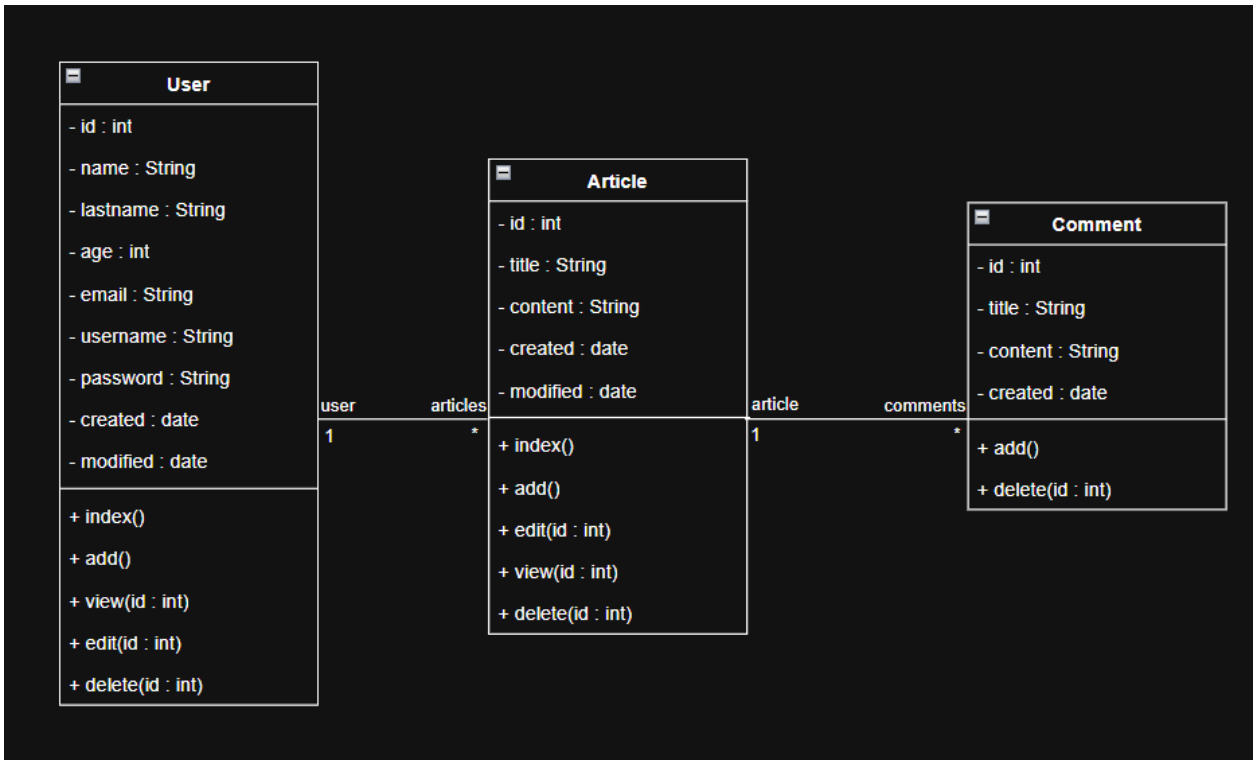
Ajout du sélecteur dans la vue add

ligne du « user_id »

```
echo $this->Form->control('user_id',['options' => $mesUsers, 'label' => 'Selectionnez un utilisateur']);
echo $this->Form->button(__("Créer l'article"));
```

3EME PARTIE — Fonctionnalités : les commentaires

Diagramme de classe Comment



Migration

Création de la table en CMD

```
C:\wamp64\www\blog_app\bin>cake bake migration CreateComments title:string content:string created article_id:int

Creating file C:\wamp64\www\blog_app\config\Migrations\20251105042828_CreateComments.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251105042828_CreateComments.php`
```

Le article_id sera en « string » dans le fichier de migration car il ne reconnaît pas int (qui doit être nommé par Integer) il faut le changer puis garder uniquement « null = false » dans les propriétés des colonnes du fichier de migrations

```
C:\wamp64\www\blog_app\bin>cake migrations migrate

 == 20251105042828 CreateComments: migrating
 == 20251105042828 CreateComments: migrated 1.7994s
All Done. Took 2.6609s
```

Faire la migration

Ajout de la clé étrangère dans la migration

Ajouter « addForeignKey » puis cake migrations rollback, ensuite cake migrations migrate dans le CMD

```
 */
public function change(): void
{
    $table = $this->table('comments');
    $table->addColumn('title', 'string', [
        'default' => null,
        'limit' => 255,
        'null' => false,
    ]);
    $table->addColumn('content', 'string', [
        'default' => null,
        'limit' => 255,
        'null' => false,
    ]);
    $table->addColumn('created', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addColumn('article_id', 'integer', [
        'null' => false,
    ]);
    $table->addForeignKey('article_id', 'articles', 'id');
    $table->create();
}
```

Pour vérifier le fonctionnement dans phpMyAdmin (dans l'onglet Structure)

	#	Nom	Type	Interclassement	Attributs	Null	Valeur par défaut	Commentaires	Extra	Action		
☐	1	**id**	int			Non	*Aucun(e)*		AUTO_INCREMENT	Modifier	Supprimer	Plus
☐	2	**title**	varchar(255)	utf8mb4_0900_ai_ci		Non	*Aucun(e)*			Modifier	Supprimer	Plus
☐	3	**content**	varchar(255)	utf8mb4_0900_ai_ci		Non	*Aucun(e)*			Modifier	Supprimer	Plus
☐	4	**created**	datetime			Non	*Aucun(e)*			Modifier	Supprimer	Plus
☐	5	**article_id**	int			Non	*Aucun(e)*			Modifier	Supprimer	Plus

– Validateur et liaison belongsTo et hasMany

CommentsTable, ajouter belongsTo (*plusieurs commentaires appartiennent à un article*) et les validateurs titre, contenu et article_id :

```
class CommentsTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->belongsTo('Articles');
    }

    public function validationDefault(Validator $validator): Validator {
        $validator
                ->notEmptyString('title', __('Veuillez renseigner un titre'))
                ->notEmptyString('content', _('Veuillez renseigner une description'))
                ->notEmptyString('article_id', _('Veuillez renseigner un utilisateur'));

        return $validator;
    }
}
```

Dans ArticlesTable rajouter le hasMany (*a plusieurs*) :

```
class ArticlesTable extends Table {

    public function initialize(array $config): void {
        $this->addBehavior('Timestamp');
        $this->belongsTo('Users');
        $this->hasMany('Comments');
    }
```

Pour compter le nombre de commentaire dans un article, vue index.php dans Articles :

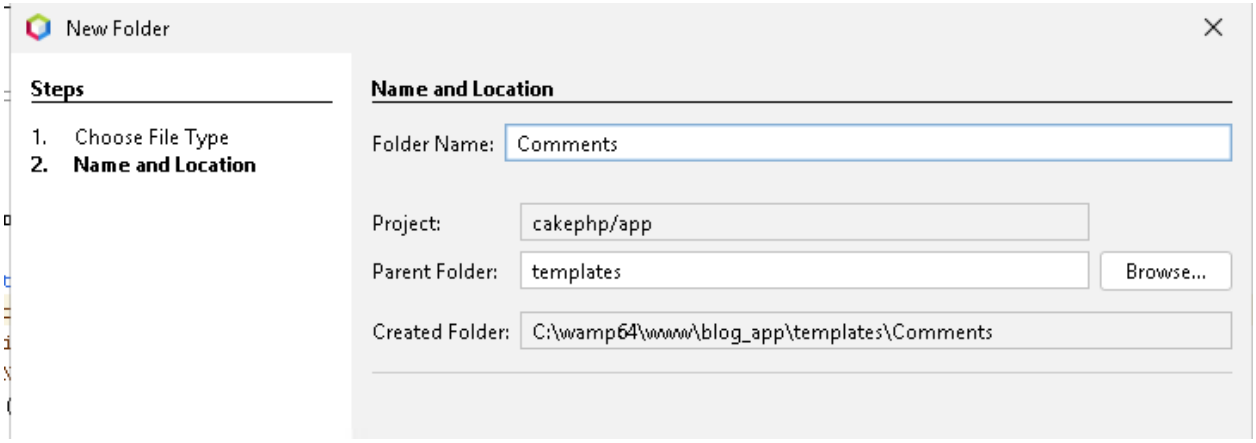
```
<td><?= count($article->comments) ?></td>
```

– Méthode add dans CommentsController

```php
 */
class CommentsController  extends AppController {

    public function add() {
        $leNewComment = $this->Comments->newEmptyEntity();
        if ($this->request->is('post')) {
            $leNewComment = $this->Articles->patchEntity($leNewComment, $this->request->getData());
            if ($this->Comments->save($leNewComment)) {
                $this->Flash->success(__("Le commentaire a été sauvegardé."));
                return $this->redirect(['action' => 'add']);
            } else {
                $this->Flash->error(__("Impossible d'ajouter le commentaire."));
            }
        }
        $this->set(compact('leNewComment'));
    }
}
```

– Création de la vue comments

Il manque la vue, il faut créer un dossier « Comments »



Création d'un fichier comments.php. Dans la vue add.php de Comments :

```php
<h1>Ajouter un commentaire</h1>
<?php
    echo $this->Form->create($leNewComment);
    echo $this->Form->control('title');
    echo $this->Form->control('content');

    echo $this->Form->button(__("Sauvegarder le commentaire"));
    echo $this->Form->end();
    echo $this->html->link('Retour à la liste des articles', ['controller' => 'articles', 'action' => 'index'],['class' => 'button']);
```

- Les commentaires dans ArticlesController

On souhaite ajouter et afficher les commentaires directement dans les articles.

Dans le controller ArticlesController, fonction détail :

```
public function detail($id = null) {
    $comments = $this->fetchTable('Comments');
    $leNewComment = $comments->newEmptyEntity();
    try {
        $leArticle = $this->Articles->get($id);
    } catch (\Exception $ex) {
        if ($id == null) {
            $this->Flash->error(__("L'action detail doit être appelé avec un identifiant"));
        } else {
            $this->Flash->error(__("L'article {0} n'existe pas", $id));
        }
        return $this->redirect(['action' => 'index']);
    }

    $this->set(compact('leArticle', 'leNewComment'));
}
```

On rajoute $leNewComment = $comments->newEmptyEntity() ; ainsi que dans le set compact

On ajoute dans ArticlesController.php dans détail, le code de la fonction add() de CommentsController.php après newEmptyEntity() ;

```
public function detail($id = null) {
    $comments = $this->fetchTable('Comments');
    $leNewComment = $comments->newEmptyEntity();

    if ($this->request->is('post')) {
        $leNewComment = $comments->patchEntity($leNewComment, $this->request->getData());
        $leNewComment->article_id = $id ;
        if ($comments->save($leNewComment)) {
            $this->Flash->success(__("Le commentaire a été sauvegardé."));
            return $this->redirect(['action' => 'detail', $id]);
        } else {
            $this->Flash->error(__("Impossible d'ajouter le commentaire."));
        }
    }

    try {
        $leArticle = $this->Articles->get($id);
    } catch (\Exception $ex) {
        if ($id == null) {
            $this->Flash->error(__("L'action detail doit être appelé avec un identifiant"));
        } else {
            $this->Flash->error(__("L'article {0} n'existe pas", $id));
        }
        return $this->redirect(['action' => 'index']);
    }

    $this->set(compact('leArticle', 'leNewComment'));
}
```

Dans le try, on rajoute le contain à côté de $id dans la variable $leArticle :

```
try {
    ## test si un $id d'article est existant
    $leArticle = $this->Articles->get($id, contain: ['Comments'=> function ($q) {
                        return $q
                                    ->order(['Comments.created asc']);
                    }]);
} catch (\Exception $ex) {
    if ($id == null) {
        $this->Flash->error(__("L'action detail doit être appelé avec un identifiant"));
    } else {
        $this->Flash->error(__("L'article {0} n'existe pas", $id));
    }
    return $this->redirect(['action' => 'index']);
}

$this->set(compact('leArticle', 'leNewComment'));
```

- Méthode delete dans CommentsController

Dans CommentsController.php on ajoute la fonction delete() :

```
public function delete($id = null) {
    $this->request->allowMethod(['post', 'delete']);
    $leComment = $this->Comments->get($id);
    if ($this->Comments->delete($leComment)) {
        $this->Flash->success(__("Le commentaire {0} d' id {1} a bien été supprimé ! ", $leComment->title, $leComment->id));
    return $this->redirect(['controller' => 'articles', 'action' => 'detail', $leComment->article_id]);
    }
```

- Script d'affichage (jquery371min)

On veut afficher les commentaires directement sous un article. Dans detail.php du template Article, on ajoute le script « jquery371min » qui servira d'affichage ou non de la section d'ajout de commentaire :

```
<?php
echo $this->Html->script('jquery371min');
?>

<script>
    $(document).ready(function () {
        $("#showcom").click(function () {
            if ($("#display").is(":visible") === false)
            {
                $("#display").show();
            }
            else {
                $("#display").hide();
            }
        });
    });
</script>
```

Le script est à télécharger dans le répertoire webroot\js

- Affichage du formulaire

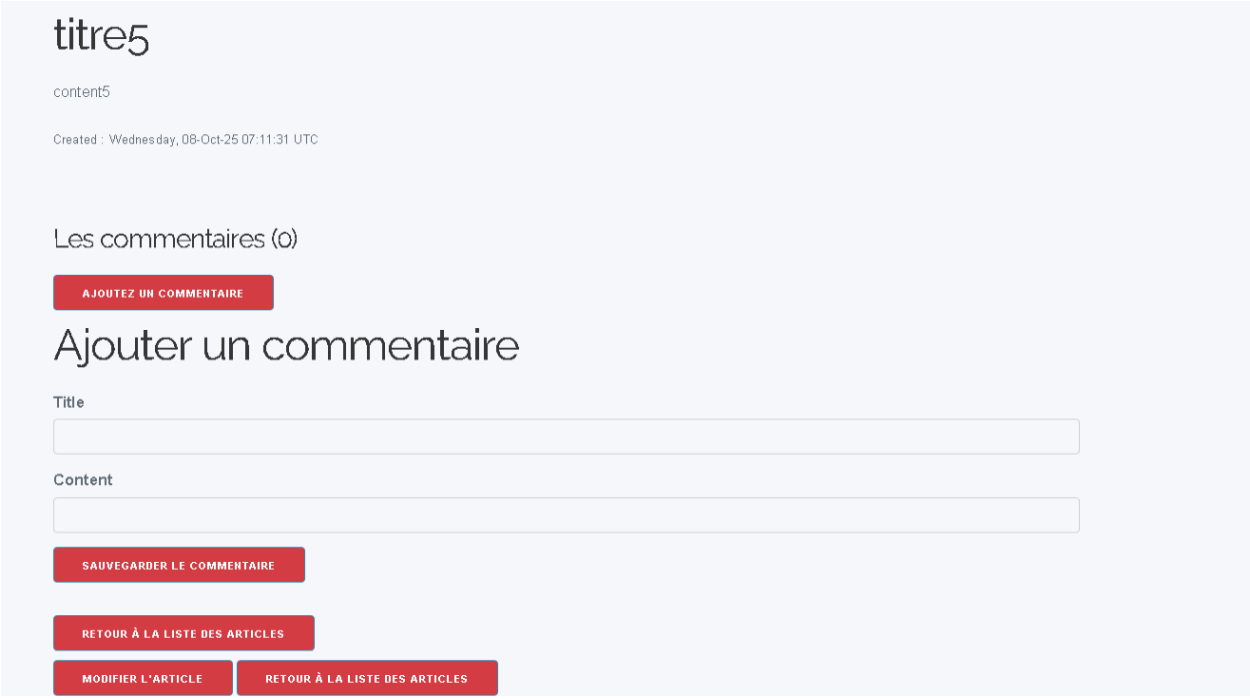
Toujours dans **detail.php** de articles on affiche le formulaire de commentaire après le script :

```
<h3>Les commentaires (<?= count($leArticle->comments) ?>) </h3>
<?php foreach ($leArticle->comments as $comm): ?>
    <table border="1">
        <tr>
            <td><?= $comm->title ?></td>
        </tr>
        <tr>
            <td><?= nl2br(h($comm->content)) ?></td>
        </tr>
        <tr>
            <td>id : <?= $comm->id ?>
                Créer le : <?= $comm->created->format(DATE_RFC850) ?>
            </td>
        </tr>
        <tr>
            <td> <?=
                $this->Form->postLink(
                        __('Supprimer'),
                        ['controller' => 'comments', 'action' => 'delete', $comm->id],
                        ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $comm->title, $comm->id),
                            'class' => 'button',
                            'style' => 'background-color :#E6101F']);
                ?>
            </td>
        </tr>
    </table>
<?php endforeach; ?>

<?=
$this->Html->link(
    'Ajoutez un commentaire', '#', ['class' => 'button', 'id' => 'showcom']
);
?>

<div id="display" style="display: none">
    <?= $this->element('comments'); ?>
</div>
```

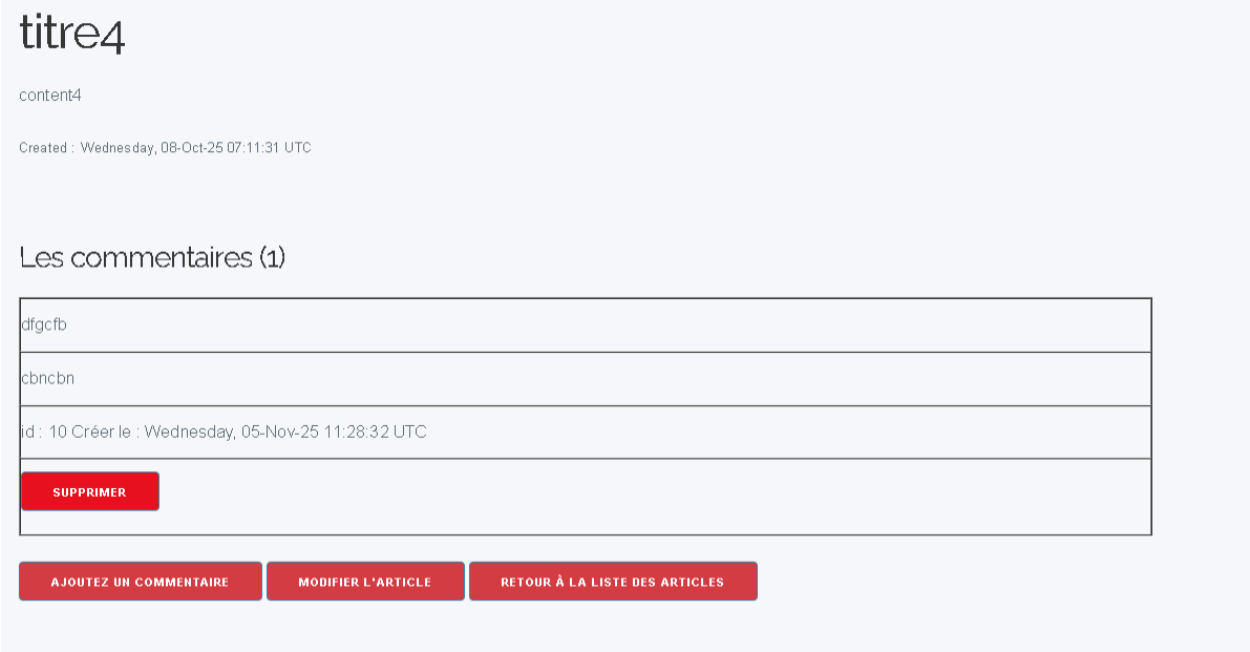
Après le formulaire on ajoute le bouton d'ajout de commentaire.

```
<?=
$this->Html->link(
    'Ajoutez un commentaire', '#', ['class' => 'button', 'id' => 'showcom']
);
?>

<div id="display" style="display: none">
    <?= $this->element('comments'); ?>
</div>
```

Le résultat sur un article :



Le bouton « retour à la liste des articles » peut être retiré s'il est en double dans le formulaire.



- Affichage de la liste des utilisateurs

Il manque les informations pour l'affichage des users…

De même pour l'affichage des utilisateurs : le script et le bouton d'affichage se retrouve avant la balise <table> des Users.

```php
<?php
echo $this->Html->script('jquery371min');
?>

<script>
    $(document).ready(function () {
        $("#showusers").click(function () {
            if ($("#displayUsers").is(":visible") === false)
            {
                $("#displayUsers").show();
            }
            else {
                $("#displayUsers").hide();
            }
        });
    });
</script>

<?=
$this->Html->link(
    'Afficher les utilisateurs', '#', ['class' => 'button', 'id' => 'showusers']
);
?>
```

```html
<div id="displayUsers" style="display: none">

<table>
    <tr>
        <th>Id</th>
        <th>Name</th>
        <th>Last Name</th>
        <th>Age</th>
        <th>Email</th>
        <th>Username</th>
        <th>Nb d'art. créés :</th>
        <th>Date de création</th>
        <th>Date de modification</th>
        <th>Action</th>
    </tr>
```

Contrairement à $this->element de Comments, il n'est pas obligé de faire un fichier element pour les utilisateurs. Il suffit de placer la balise div (ouvert et fermer) juste avant et après la balise <table>.

Le résultat :

CakePHP

Utilisateurs Articles

Tous les utilisateurs du Blog

AJOUTER UN UTILISATEUR | AFFICHER LES UTILISATEURS

CakePHP

Utilisateurs Articles Documentation API

Tous les utilisateurs du Blog

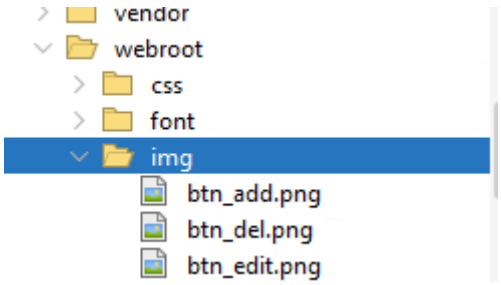
AJOUTER UN UTILISATEUR | AFFICHER LES UTILISATEURS

Id	Name	Last Name	Age	Email	Username	Nb d'art. créés :
1	TestPrenom1	TestNom1	18	test1@gmail.com	user1	5
2	TestPrenom2	TestNom2	1	test@gmail.com	user2	0

– Boutons remplacés par des icônes

On remplace les boutons principaux d'ajout, d'édition ou de suppression d'articles/d'utilisateurs en ajoutant des icônes :

Dans **webroot/img**, on ajoute les images qu'on souhaite (Ajout, Supprimer, Edit) :



Dans la vue **index** de Articles et de Users.

Tous en haut après la balise <h1> :

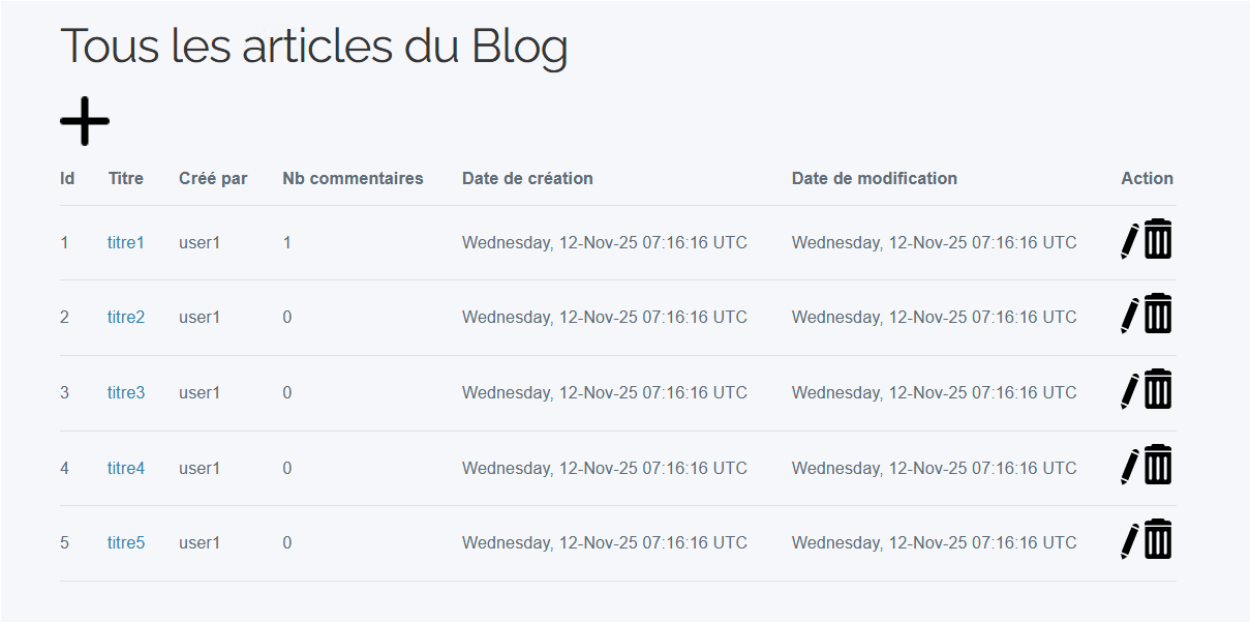
```
<?=
$this->html->image("btn_add.png",["alt" => "Add", 'url' => ['action' => 'add'], 'style' => 'height:48px;']);
?>
```

Dans le tableau après created et modified :

```
<td> <?= $this->html->link($this->html->image("btn_edit.png", ["alt" => "edit"]),
            ['action' => 'edit', $user->id,
            'style' => 'height:48px;'],
            ['escape' => false]);
    ?>
    <?=
    $this->Form->postLink($this->html->image("btn_del.png", ["alt" => "del"]),
            ['action' => 'delete', $user->id],
            ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $user->name, $user->id), 'escape' => false]);
    ?>
</td>
```

<u>UPDATE</u> *: il faut retirer le 'url =>' dans les html->link et plus précisément avant action*

Le résultat sur la page articles :



- Onglets dans la navbar

Informations à compléter avec des screens

Dans **template** > **layout** > **default.php** :

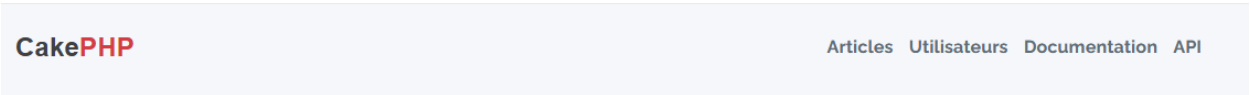
On retrouve les liens placés dans la barre de navigation dans la balise <nav> :

```
<body>
    <nav class="top-nav">
        <div class="top-nav-title">
            <a href="<?= $this->Url->build('/') ?>"><span>Cake</span>PHP</a>
        </div>
        <div class="top-nav-links">
            <a href="<?= $this->Url->build('/users') ?>">Utilisateurs</a>
            <a target="_blank" rel="noopener" href="https://book.cakephp.org/5/">Documentation</a>
            <a target="_blank" rel="noopener" href="https://api.cakephp.org/">API</a>
        </div>
    </nav>
    <main class="main">
        <div class="container">
            <?= $this->Flash->render() ?>
            <?= $this->fetch('content') ?>
        </div>
    </main>
    <footer>
    </footer>
</body>
</html>
```

On ajoute un nouvel onglet avec :

<a href </a> avec $this->Url->build('/nomdelapage') pour permettre le lien des pages.

Le résultat :



On peut faire le lien dans la navbar en forme de bouton :

```
echo '<span>';
echo $this->Html->link('Connexion', [
    'controller' => 'Users',
    'action' => 'login'
], ['class' => 'button', 'style' => 'background-color : blue;']);
```

4E PARTIE – SÉCURISATION ET CONNEXION

➢ Procédure CakePHP 3.0

Procédure sur le site de Cake PHP :

https://book.cakephp.org/authentication/3/fr/index.html

Dans src/application.php :

```
namespace App;

use Cake\Core\Configure;
use Cake\Core\ContainerInterface;
use Cake\Datasource\FactoryLocator;
use Cake\Error\Middleware\ErrorHandlerMiddleware;
use Cake\Http\BaseApplication;
use Cake\Http\Middleware\BodyParserMiddleware;
use Cake\Http\Middleware\CsrfProtectionMiddleware;
use Cake\Http\MiddlewareQueue;
use Cake\ORM\Locator\TableLocator;
use Cake\Routing\Middleware\AssetMiddleware;
use Cake\Routing\Middleware\RoutingMiddleware;
use Authentication\AuthenticationService;
use Authentication\AuthenticationServiceInterface;
use Authentication\AuthenticationServiceProviderInterface;
use Authentication\Identifier\AbstractIdentifier;
use Authentication\Identifier\IdentifierInterface;
use Authentication\Middleware\AuthenticationMiddleware;
use Cake\Routing\Router;
use Psr\Http\Message\ServerRequestInterface;
```

Plus en bas, ajouter implements AuthenticationServiceProviderInterface

```
class Application extends BaseApplication implements AuthenticationServiceProviderInterface {
    /**
```

```php
public function middleware(MiddlewareQueue $middlewareQueue): MiddlewareQueue
{
    $middlewareQueue
        // Catch any exceptions in the lower layers,
        // and make an error page/response
        ->add(new ErrorHandlerMiddleware(Configure::read('Error'), $this))

        // Handle plugin/theme assets like CakePHP normally does.
        ->add(new AssetMiddleware([
            'cacheTime' => Configure::read('Asset.cacheTime'),

        // Add the AuthenticationMiddleware. It should be
    // after routing and body parser.

        ]))
        // Add routing middleware.
        // If you have a large number of routes connected, turning on routes
        // caching in production could improve performance.
        // See https://github.com/CakeDC/cakephp-cached-routing
        ->add(new RoutingMiddleware($this))

        // Parse various types of encoded request bodies so that they are
        // available as array through $request->getData()
        // https://book.cakephp.org/5/en/controllers/middleware.html#body-parser-middleware
        ->add(new BodyParserMiddleware())
      // Add the AuthenticationMiddleware. It should be
          // after routing and body parser.
        ->add(new AuthenticationMiddleware($this))
        // Cross Site Request Forgery (CSRF) Protection Middleware
        // https://book.cakephp.org/5/en/security/csrf.html#cross-site-request-forgery-csrf-middleware
        ->add(new CsrfProtectionMiddleware([
            'httponly' => true,
        ]));
```

Ajouter AuthenticationMiddleware en dessous de BodyParser

```php
// https://book.cakephp.org/5/en/controllers/middleware.html#body-parser-m
->add(new BodyParserMiddleware())
// Add the AuthenticationMiddleware. It should be
    // after routing and body parser.
->add(new AuthenticationMiddleware($this))
// Cross Site Request Forgery (CSRF) Protection Middleware
// https://book.cakephp.org/5/en/security/csrf.html#cross-site-request-for
```

Ajouter la méthode AuthenticationService :

```php
public function getAuthenticationService(ServerRequestInterface $request): AuthenticationServiceInterface
{
    $service = new AuthenticationService();

    // Définissez vers où les utilisateurs doivent être redirigés s'ils ne
    // sont pas authentifiés
    $service->setConfig([
        'unauthenticatedRedirect' => Router::url([
                'prefix' => false,
                'plugin' => null,
                'controller' => 'Users',
                'action' => 'login',
        ]),
        'queryParam' => 'redirect',
    ]);

    $fields = [
        AbstractIdentifier::CREDENTIAL_USERNAME => 'email',
        AbstractIdentifier::CREDENTIAL_PASSWORD => 'password'
    ];
    // Chargez les authentificateurs. Session est censé figurer en premier.
    $service->loadAuthenticator('Authentication.Session');
    $service->loadAuthenticator('Authentication.Form', [
        'fields' => $fields,
        'loginUrl' => Router::url([
            'prefix' => false,
            'plugin' => null,
            'controller' => 'Users',
            'action' => 'login',
        ]),
    ]);

    // Chargez les identificateurs
    $service->loadIdentifier('Authentication.Password', compact('fields'));
```

Activer Windows

A la fin de la méthode : return $service ;

Dans AppController.php, rajouter loadComponent et allowUnauthenicated :

```php
class AppController extends Controller
{
    /**
     * Initialization hook method.
     *
     * Use this method to add common initialization code like loading components.
     *
     * e.g. `$this->loadComponent('FormProtection');`
     *
     * @return void
     */
    public function beforeFilter(\Cake\Event\EventInterface $event)
    {
        $this->Authentication->allowUnauthenticated(['view', 'index', 'detail']);
    }

    public function initialize(): void
    {
        parent::initialize();

        $this->loadComponent('Flash');

        $this->loadComponent('Authentication.Authentication');
```

➢ Action login

Ajout du BeforeFilter dans UsersController.php

```
    }
    public function beforeFilter(\Cake\Event\EventInterface $event)
    {
        parent::beforeFilter($event);
        // Allow users to register and logout.
        // You should not add the "login" action to allow list. Doing so would
        // cause problems with normal functioning of AuthComponent.
        $this->Authentication->allowUnauthenticated(['login', 'add']);
    }
```

Ajout de la fonction login dans UsersController :

```
    public function login()
    {
        $result = $this->Authentication->getResult();
        // Si l'utilisateur est connecté, le renvoyer ailleurs
        if ($result && $result->isValid()) {
            $target = $this->Authentication->getLoginRedirect() ?? '/';
            return $this->redirect($target);
        }
        if ($this->request->is('post')) {
            $this->Flash->error('Identifiant ou mot de passe invalide');
    }
    }
```

➢ Vue login – UsersController

- Ancien version – Cake 3.0

```
<div class="users form content">

    <?= $this->Form->create() ?>

    <fieldset>

        <legend><?= __('Saisissez votre identifiant et votre mot de passe svp')
?></legend>

        <?= $this->Form->control('email') ?>

        <?= $this->Form->control('password') ?>

    </fieldset>

    <?= $this->Form->button(__('Login')); ?>

    <?= $this->Form->end() ?>

</div>
```

- Nouvelle version

HTML

```php
<div class="login-container">
    <!-- <div class="users form content"> -->
        <div class="card">
            <div class="card-header">
                <h2 class="text-center"><?= __('Connexion') ?></h2>
            </div>
            <div class="card-body">
                <?= $this->Form->create() ?>
                <fieldset>
                    <!--<legend class="text-muted text-center"><?= __('Veuillez saisir votre email et mot de passe') ?></legend>

                    <div class="form-group">
                        <?=
                        $this->Form->control('email', [
                            'label' => __(''),
                            'class' => 'form-control',
                            'placeholder' => __('votre@email.com')
                        ])
                        ?>
                    </div>

                    <div class="form-group">
                        <?=
                        $this->Form->control('password', [
                            'label' => __(''),
                            'class' => 'form-control',
                            'placeholder' => __('Mot de passe')
                        ])
                        ?>
                    </div>
                </fieldset>

                <div class="form-group text-center">
                <?=
                $this->Form->button(__('Se connecter'), [
                    'class' => 'btn btn-primary btn-lg btn-block'
                ])
```

Activer Windows

```php
                ?>
                </div>
                </fieldset>

                <div class="form-group text-center">
                <?=
                $this->Form->button(__('Se connecter'), [
                    'class' => 'btn btn-primary btn-lg btn-block'
                ])
                ?>
                </div>

                <div class="form-group text-center">
                <?=
                $this->Html->link("S'inscrire", [
                        'controller' => 'Users',
                        'action' => 'add'
                    ]);
                ?>
                </div>

                <?= $this->Form->end() ?>
            </div>
        </div>
    </div>
</div>
```

Style :

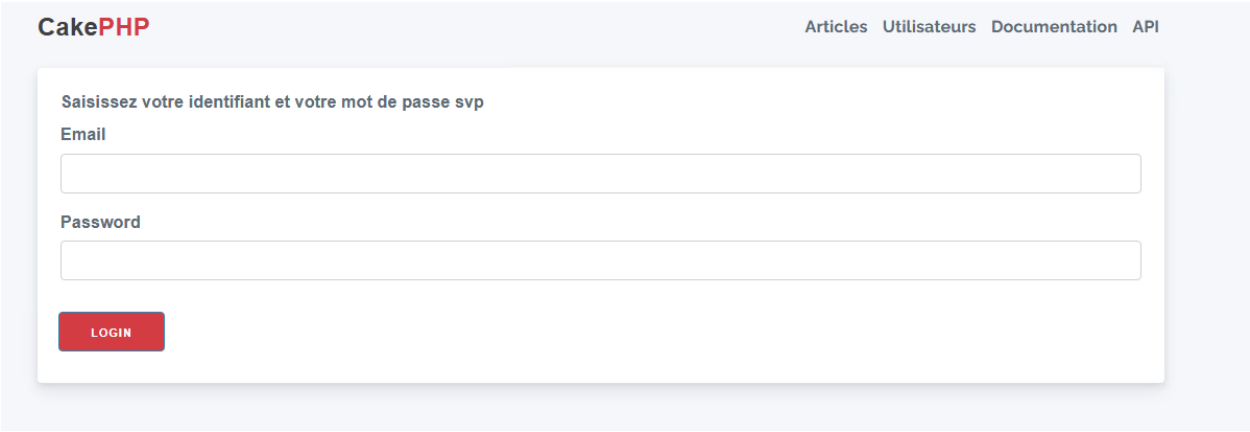
```style
<style>
    .login-container {
        display: flex;
        justify-content: center;
        align-items: center;
        min-height: 80vh;
        padding: 20px;
        background-color: #f8f9fa;
    }

    .users.form.content {
        width: 100%;
        max-width: 400px;
    }

    .card {
        border: none;
        border-radius: 10px;
        box-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
        overflow: hidden;
        width: 100%;
        max-width: 400px;

    }

    .card-header {
        background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        color: white;
        padding: 2rem 1rem;
        text-align: center;
    }
```

```css
.card-header h2 {
    margin: 0;
    font-weight: 800;
    font-size: 2rem;
    color: white;
}

.card-body {
    padding: 2rem;
    background: white;
}

.form-group {
    margin-bottom: 1.5rem;
}

.form-control {
    border-radius: 5px;
    border: 1px solid #ddd;
    padding: 12px 15px;
    font-size: 14px;
    transition: all 0.3s;
}

.form-control:focus {
    border-color: #667eea;
    box-shadow: 0 0 0 0.2rem rgba(102, 126, 234, 0.25);
}
```

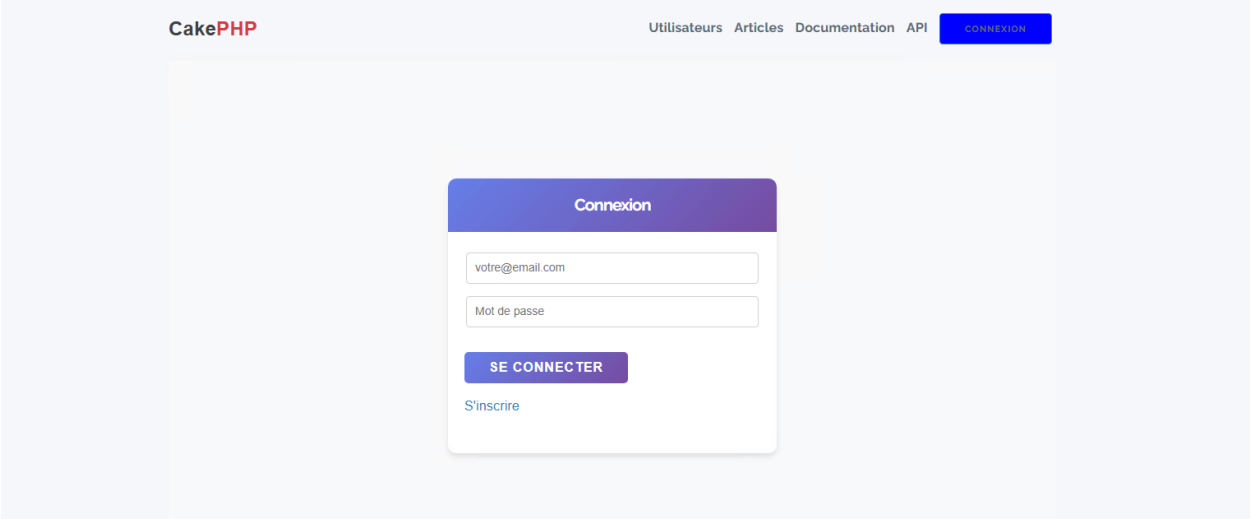
```css
.btn-primary {
    display:flex;
    justify-content:center;
    align-items:center;
    background: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
    border: none;
    border-radius: 5px;
    padding: 12px 30px;
    font-size: 16px;
    transition: all 0.3s;

}

.btn-primary:hover {
    transform: translateY(-2px);
    box-shadow: 0 4px 8px rgba(102, 126, 234, 0.3);
}

legend {
    font-size: 14px;
    margin-bottom: 1.5rem;
    border: none;
}

.text-muted {
    color: #6c757d !important;
}

/* Responsive */
@media (max-width: 576px) {
    .card-body {
        padding: 1.5r  em;
    }
```

```
        .login-container {
            min-height: 70vh;
        }
    }
</style>
```

Le résultat Cake 3.0:



Le résultat Nouvelle version :



➢ Utilisateur attribué à un article (auto)

Quand un utilisateur est connecté il faut faire en sorte que l'utilisateur puisse ajouter ou modifier un article qui est publié ou modifié avec **son username**

Vue add, on commente la ligne de la sélection du user_id
(*qui permettait de choisir manuellement*) :

```
//echo $this->Form->control('user_id',['options' => $mesUsers, 'label' => 'Selectionnez un utilisateur']);
```

ArticlesController : ajouter la variable *$leNewArticle->user_id* qui permet de sauvegarder l'utilisateur de l'article

```
if ($this->request->is('post')) {
    $leNewArticle = $this->Articles->patchEntity($leNewArticle, $this->request->getData());
    $leNewArticle->user_id = $this->request->getAttribute('identity')->id;
    if ($this->Articles->save($leNewArticle)) {
```

Il faut pouvoir ajouter le nom de l'utilisateur dans l'article créé par ce même utilisateur. On doit faire un contain.

ArticlesController, action detail, il faut rajouter :

```
        ],
        'Users' => function ($q) {
            return $q
                ->select(['Users.username']);
        }
    ]);
```

Vue détail, rajouter dans le paragraphe <p> en dessous de la date :

```
<small>Auteur : <?= $leArticle->user->username ?></small>
```

Le résultat :

Created : Wednesday, 12-Nov-25 07:16:16 UTC

Auteur : user1

Association des utilisateurs aux commentaires

De même pour les Commentaires :

- Lier les modèles : Users et Comments
  - Users hasMany Comments
  - Comments belongsTo Users
- Modification BDD
  - Fichier de migration
    - Créer FK user_id -> Comments
    - Renseigner une valeur user_id grâce à un update
- Modifier ArticlesController
  - Fonction detail
    - Prendre en compte user_id

Lier les modèles :

- Dans UsersTable, ajouter Users hasMany Comments :

```
public function initialize(array $config): void {
    $this->addBehavior('Timestamp');
    $this->hasMany('Articles');
    $this->hasMany('Comments');
}
```

- Dans CommentsTable, ajouter Comments belongsTo Users :

```
public function initialize(array $config): void {
    $this->addBehavior('Timestamp');
    $this->belongsTo('Articles');
    $this->belongsTo('Users');
}
```

Modification BDD :

- Création du fichier de migration AlterCommentsAddFkUsersId
- Créer la colonne user_id
- Faire une UPDATE pour pouvoir remplir tous les champs vide

- Ajouter la clé étrangère

```php
class AlterCommentsAddFkUsersId extends BaseMigration
{
    /**
     * Change Method.
     *
     * More information on this method is available here:
     * https://book.cakephp.org/migrations/4/en/migrations.html#the-change-method
     *
     * @return void
     */
    public function up(): void
    {
        $table = $this->table('comments');
        $table->addColumn('user_id', 'integer', [
            'null' => false
        ]);
        $table->update();

        $this->execute('UPDATE comments SET user_id = (SELECT id from users LIMIT 1)');

        $table2 = $this->table('comments');
        $table2->addForeignKey('user_id', 'users', 'id');
        $table2->update();
    }

    public function down(): void
    {
        $table = $this->table('comments');
        $table->dropForeignKey('user_id');
        $table->removeColumn('user_id');
        $table->update();
    }
}
```

Modifier ArticlesController :

- Dans la fonction detail :
  - 1. Ajouter la variable qui permet d'obtenir l'attribut identity lorsque l'utilisateur est connecté.
  - 2. Joindre l'utilisateur dans le contain de $leArticle

1 – Mettre la variable dans le if du request post.

```php
$leNewComment->article_id = $id;
$leNewComment->user_id = $this->request->getAttribute('identity')->id;
```

– Sécuriser l'intégrité des données

Dans IndexArticles :

- ➢ Quand l'utilisateur est connecté :
- Afficher les boutons edit et supprimer que par les article du user logué.

```
<td> <?php $identity = $this->request->getAttribute('identity');
if ($identity && ($article->user_id == $identity->id)) {
echo


} ?>
</td>
```

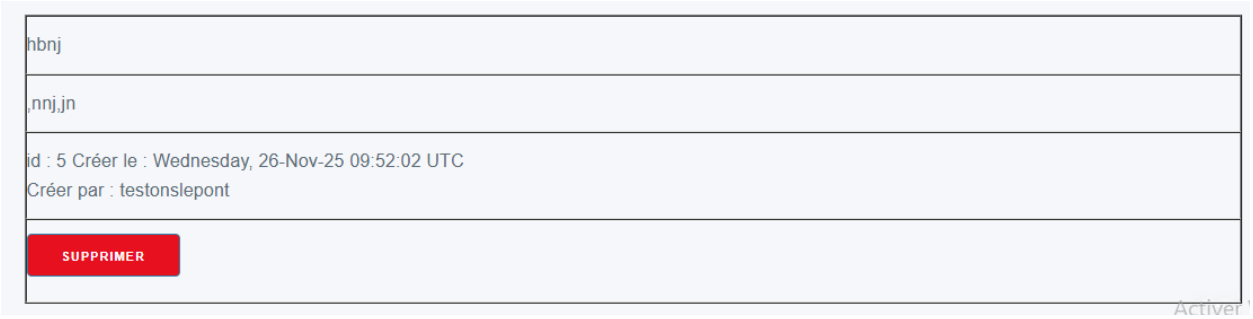
```
<td> <?php $identity = $this->request->getAttribute('identity');
if ($identity && ($article->user_id == $identity->id)) {
echo
$this->html->link($this->html->image("btn_edit.png", ["alt" => "edit"]),
            ['action' => 'edit', $article->id,
            'style' => 'height:48px;'],
            ['escape' => false]);

echo $this->Form->postLink($this->html->image("btn_del.png", ["alt" => "del"]),
            ['action' => 'delete', $article->id],
            ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $article->name, $article->id), 'escape' => false]);

} ?>
</td>
```

9	gjvhjhjvj	test	0	Wednesday, 26-Nov-25 03:36:44 UTC	Wednesday, 26-Nov-25 04:06:18 UTC
10	yffygyhj	test	0	Wednesday, 26-Nov-25 04:08:18 UTC	Wednesday, 26-Nov-25 04:11:47 UTC
11	t	testonslepont	0	Wednesday, 26-Nov-25 09:26:05 UTC	Wednesday, 26-Nov-25 09:26:05 UTC

```
try {
    $leArticle = $this->Articles->get($id);
    $identity = $this->request->getAttribute('identity');

    if ($identity && ($leArticle->user_id != $identity->id)) {
        $this->Flash->error(__("Vous n'avez pas le droit d'éditer cet article"));
        return $this->redirect(['action' => 'index']);
    }
```

Vous n'avez pas le droit d'éditer cet article

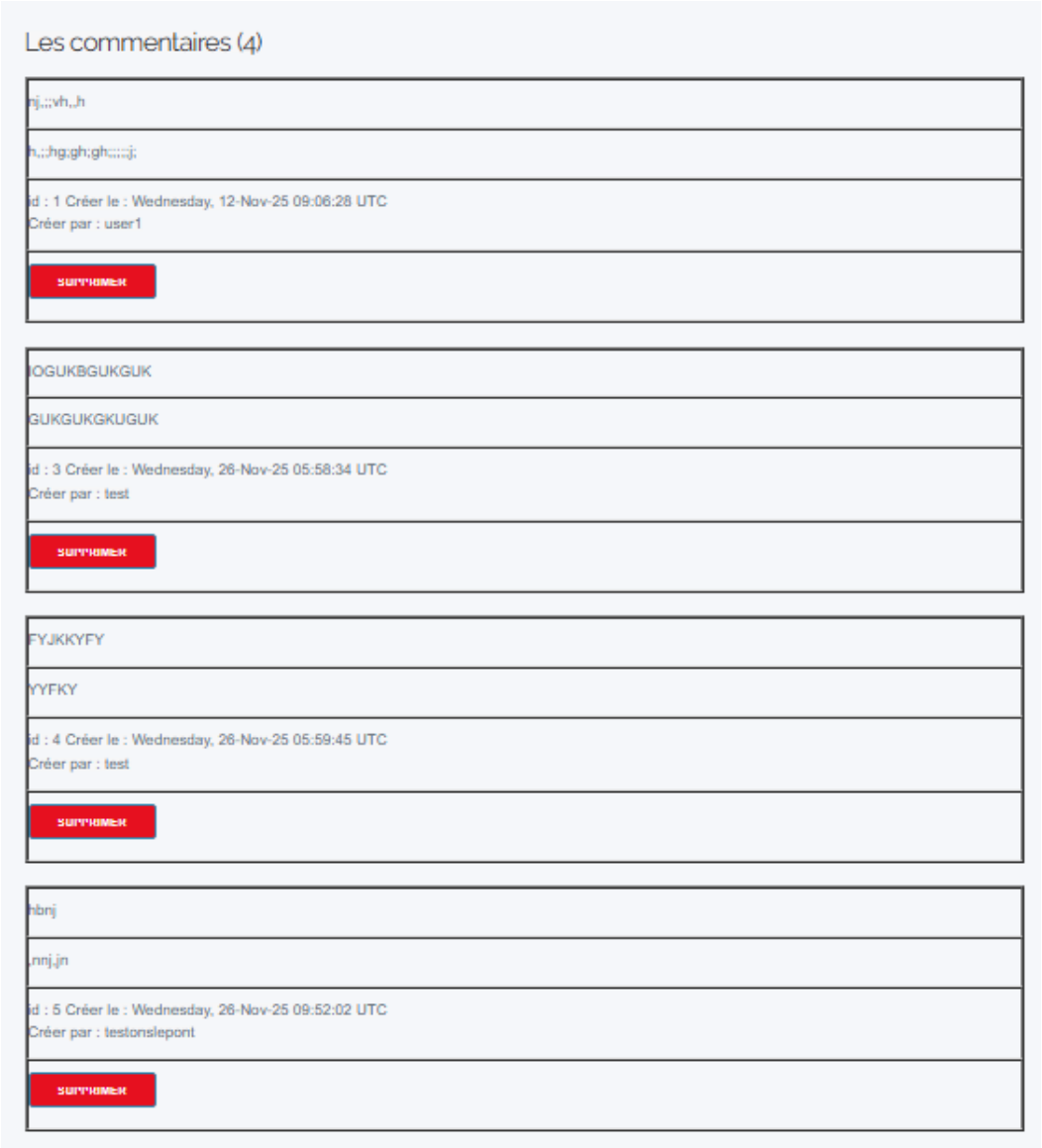
```
<tr>
    <td> <?php
    $identity = $this->request->getAttribute('identity');
    if ($identity && ($comm->user_id == $identity->id || $leArticle->user_id == $identity->id)) {
        echo
        $this->Form->postLink(
                __('Supprimer'),
                ['controller' => 'comments', 'action' => 'delete', $comm->id],
                ['confirm' => __("Vraiment supprimer {0} dont l'id vaut {1} ", $comm->title, $comm->id),
                    'class' => 'button',
                    'style' => 'background-color :#E6101F']);
    } ?>
```



Tous les commentaires qui peuvent être supprimé par l'auteur du commentaire.

Le bouton n'est plus visible quand le commentaire ne l'appartient pas

Les commentaires qui peuvent être supprimés par l'auteur de **<u>l'article</u>**



5EME PARTIE – CREATION DES TAGS

➢ Ajout des tags sur les articles

- Création fichier de migrations

```
C:\wamp64\www\blog_app\bin>cake migrations create TagsAndArticlesTags

Creating file C:\wamp64\www\blog_app\config\Migrations\20251126103238_TagsAndArticlesTags.php
Wrote `C:\wamp64\www\blog_app\config\Migrations\20251126103238_TagsAndArticlesTags.php`
```

- Dans le fichier de migration
  - Créations de la table tags

```
public function change(): void
{
    $table = $this->table('tags');
    $table->addColumn('title', 'string', [
        'default' => null,
        'limit' => 255,
        'null' => false,
    ]);
    $table->addColumn('created', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addColumn('modified', 'datetime', [
        'default' => null,
        'null' => false,
    ]);
    $table->addIndex('title', ['unique' => true]);
    $table->create();
```

Action index – TagsController

```
 */
public function index()
{
    $query = $this->Tags->find()->contain(['Articles' => function ($q) {
        return $q
                ->select(['ArticlesTags.tag_id']);
    }
    ]);
    $tags = $this->paginate($query);

    $this->set(compact('tags'));
}
```

Action view – TagsController

```
/**
 * View method
 *
 * @param string|null $id Tag id.
 * @return \Cake\Http\Response|null|void Renders view
 * @throws \Cake\Datasource\Exception\RecordNotFoundException When record not found.
 */
public function view($id = null)
{
    $tag = $this->Tags->get($id, contain: ['Articles', 'ArticlesTags']);
    $this->set(compact('tag'));
}
```

➢ Action add – TagsController

```
*/
public function add()
{
    $tag = $this->Tags->newEmptyEntity();
    if ($this->request->is('post')) {
        $tag = $this->Tags->patchEntity($tag, $this->request->getData());
        if ($this->Tags->save($tag)) {
            $this->Flash->success(__('The tag a été sauvegardé.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The tag could not be saved. Please, try again.'));
    }
    // $articles = $this->Tags->Articles->find('list', limit: 200)->all();
    $this->set(compact('tag'));
}
```

➢ Action edit – TagsController

```
public function edit($id = null)
{
    $tag = $this->Tags->get($id, contain: ['Articles']);
    if ($this->request->is(['patch', 'post', 'put'])) {
        $tag = $this->Tags->patchEntity($tag, $this->request->getData());
        if ($this->Tags->save($tag)) {
            $this->Flash->success(__('The tag has been saved.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The tag could not be saved. Please, try again.'));
    }
    // $articles = $this->Tags->Articles->find('list', limit: 200)->all();
    $this->set(compact('tag'));
}
```

➢ Action delete – TagsController

```
*/
public function delete($id = null)
{
    $this->request->allowMethod(['post', 'delete']);
    $tag = $this->Tags->get($id);
    if ($this->Tags->delete($tag)) {
        $this->Flash->success(__('The tag has been deleted.'));
    } else {
        $this->Flash->error(__('The tag could not be deleted. Please, try again.'));
    }

    return $this->redirect(['action' => 'index']);
}
```

➢ Relations et Validations – TagsTable

- Ajout des relations et clés primaire

```php
class TagsTable extends Table
{
    /**
     * Initialize method
     *
     * @param array<string, mixed> $config The configuration for the Table.
     * @return void
     */
    public function initialize(array $config): void
    {
        parent::initialize($config);

        $this->setTable('tags');
        $this->setDisplayField('title');
        $this->setPrimaryKey('id');

        $this->addBehavior('Timestamp');

        $this->hasMany('ArticlesTags', [
            'foreignKey' => 'tag_id',
        ]);
        $this->belongsToMany('Articles', [
            'foreignKey' => 'tag_id',
            'targetForeignKey' => 'article_id',
            'joinTable' => 'articles_tags',
        ]);
    }
```

- Validateurs de la table

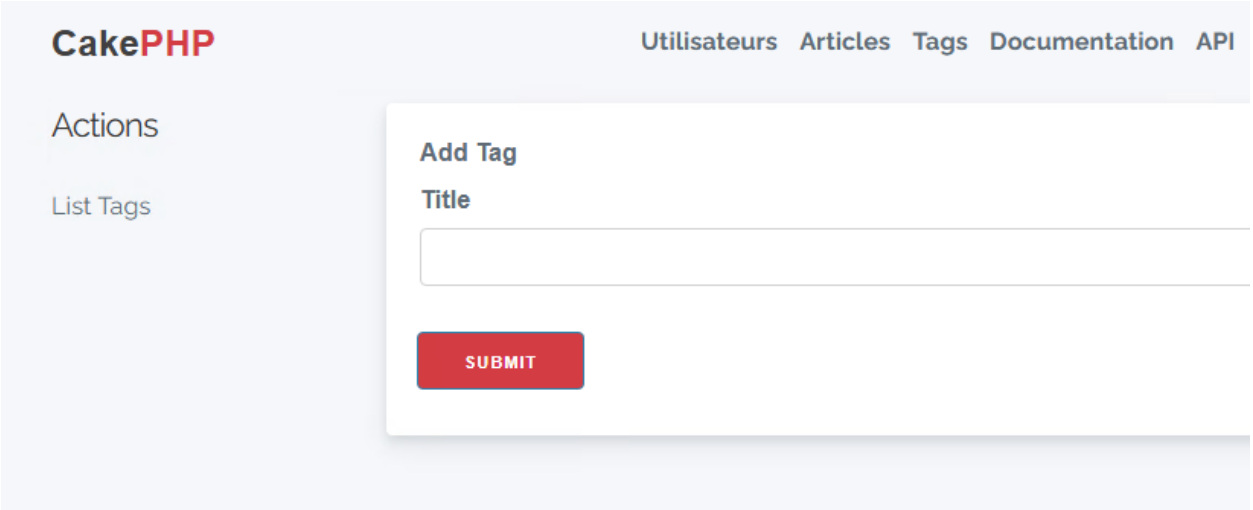
```php
    */
    public function validationDefault(Validator $validator): Validator
    {
        $validator
            ->scalar('title')
            ->maxLength('title', 255)
            ->requirePresence('title', 'create')
            ->notEmptyString('title')
            ->add('title', 'unique', ['rule' => 'validateUnique', 'provider' => 'table']);

        return $validator;
    }

    /**
     * Returns a rules checker object that will be used for validating
     * application integrity.
     *
     * @param \Cake\ORM\RulesChecker $rules The rules object to be modified.
     * @return \Cake\ORM\RulesChecker
     */
    public function buildRules(RulesChecker $rules): RulesChecker
    {
        $rules->add($rules->isUnique(['title']), ['errorField' => 'title']);

        return $rules;
    }
```

➢ Vue Tags – Template Tags

- Action Add

```php
<?php
/**
 * @var \App\View\AppView $this
 * @var \App\Model\Entity\Tag $tag
 * @var \Cake\Collection\CollectionInterface|string[] $articles
 */
?>
<div class="row">
    <aside class="column">
        <div class="side-nav">
            <h4 class="heading"><?= __('Actions') ?></h4>
            <?= $this->Html->link(__('List Tags'), ['action' => 'index'], ['class' => 'side-nav-item']) ?>
        </div>
    </aside>
    <div class="column column-80">
        <div class="tags form content">
            <?= $this->Form->create($tag) ?>
            <fieldset>
                <legend><?= __('Add Tag') ?></legend>
                <?php
                    echo $this->Form->control('title');
                    // echo $this->Form->control('articles._ids', ['options' => $articles]);
                ?>
            </fieldset>
            <?= $this->Form->button(__('Submit')) ?>
            <?= $this->Form->end() ?>
        </div>
    </div>
</div>
```

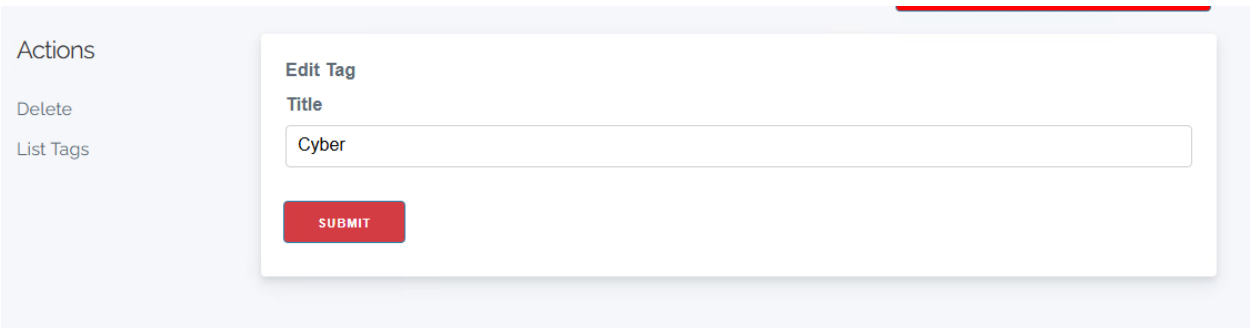
- **Le résultat :**



- Action edit

```php
<?php
/**
 * @var \App\View\AppView $this
 * @var \App\Model\Entity\Tag $tag
 * @var string[]|\Cake\Collection\CollectionInterface $articles
 */
?>
<div class="row">
    <aside class="column">
        <div class="side-nav">
            <h4 class="heading"><?= __('Actions') ?></h4>
            <?= $this->Form->postLink(
                __('Delete'),
                ['action' => 'delete', $tag->id],
                ['confirm' => __('Are you sure you want to delete # {0}?', $tag->id), 'class' => 'side-nav-item']
            ) ?>
            <?= $this->Html->link(__('List Tags'), ['action' => 'index'], ['class' => 'side-nav-item']) ?>
        </div>
    </aside>
    <div class="column column-80">
        <div class="tags form content">
            <?= $this->Form->create($tag) ?>
            <fieldset>
                <legend><?= __('Edit Tag') ?></legend>
                <?php
                    echo $this->Form->control('title');
                    // echo $this->Form->control('articles._ids', ['options' => $articles]);
                ?>
            </fieldset>
            <?= $this->Form->button(__('Submit')) ?>
            <?= $this->Form->end() ?>
        </div>
    </div>
</div>
```

- **Le résultat :**



- Action index

```php
<div class="tags index content">
    <?= $this->Html->link(__('New Tag'), ['action' => 'add'], ['class' => 'button float-right']) ?>
    <h3><?= __('Tags') ?></h3>
    <div class="table-responsive">
        <table>
            <thead>
                <tr>
                    <th><?= $this->Paginator->sort('id') ?></th>
                    <th><?= $this->Paginator->sort('title') ?></th>
                    <th> Nb Articles </th>
                    <th><?= $this->Paginator->sort('created') ?></th>
                    <th><?= $this->Paginator->sort('modified') ?></th>
                    <th class="actions"><?= __('Actions') ?></th>
                </tr>
            </thead>
            <tbody>
                <?php foreach ($tags as $tag): ?>
                <tr>
                    <td><?= $this->Number->format($tag->id) ?></td>
                    <td><?= h($tag->title) ?></td>
                    <td><?= count($tag->articles) ?></td>
                    <td><?= h($tag->created) ?></td>
                    <td><?= h($tag->modified) ?></td>
                    <td class="actions">
                        <?= $this->Html->link(__('View'), ['action' => 'view', $tag->id]) ?>
                        <?= $this->Html->link(__('Edit'), ['action' => 'edit', $tag->id]) ?>
                        <?= $this->Form->postLink(
                            __('Delete'),
                            ['action' => 'delete', $tag->id],
                            [
                                'method' => 'delete',
                                'confirm' => __('Are you sure you want to delete # {0}?', $tag->id),
                            ]
                        ) ?>
                    </td>
                </tr>
                <?php endforeach; ?>
```

Activer Windows

```php
            </tbody>
        </table>
    </div>
    <div class="paginator">
        <ul class="pagination">
            <?= $this->Paginator->first('<< ' . __('first')) ?>
            <?= $this->Paginator->prev('< ' . __('previous')) ?>
            <?= $this->Paginator->numbers() ?>
            <?= $this->Paginator->next(__('next') . ' >') ?>
            <?= $this->Paginator->last(__('last') . ' >>') ?>
        </ul>
        <p><?= $this->Paginator->counter(__('Page {{page}} of {{pages}}, showing {{current}} record(s) out of {{count}} total')) ?></p>
    </div>
</div>
```

Le résultat :

Tags

NEW TAG

Id	Title	Created	Modified	Actions
1	Cyber	11/26/25, 11:44 AM	11/26/25, 11:44 AM	View Edit Delete
2	SDSI	11/26/25, 11:48 AM	11/26/25, 11:48 AM	View Edit Delete
3	NSI	11/26/25, 11:48 AM	11/26/25, 11:48 AM	View Edit Delete
4	Math Info	11/26/25, 11:50 AM	11/26/25, 11:50 AM	View Edit Delete
6	test	11/28/25, 3:41 AM	11/28/25, 3:41 AM	View Edit Delete

< previous next >

Page 1 of 1, showing 5 record(s) out of 5 total

- Action view

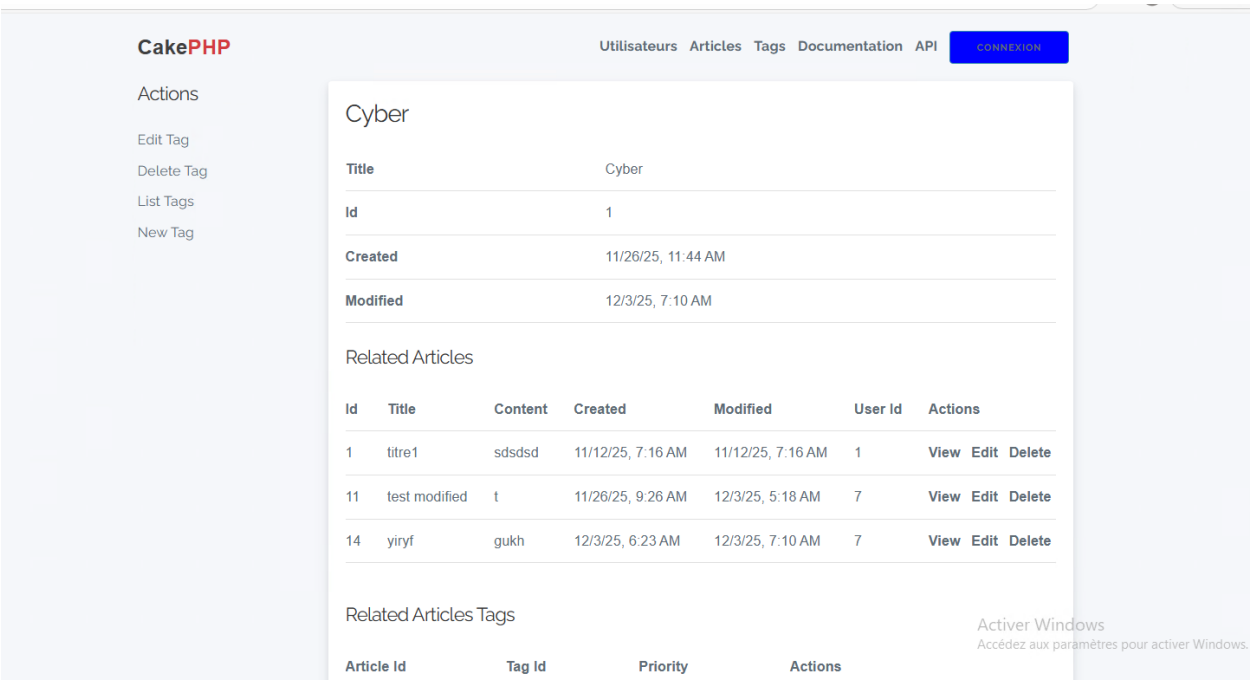
```php
<div class="row">
    <aside class="column">
        <div class="side-nav">
            <h4 class="heading">
            <?= $this->Html->link(__('Edit Tag'), ['action' => 'edit', $tag->id], ['class' => 'side-nav-item']) ?>
            <?= $this->Form->postLink(__('Delete Tag'), ['action' => 'delete', $tag->id], ['confirm' => __('Are you sure you want to delete # {0}?', $tag->id), 'class' => 'side-nav-item'])
            <?= $this->Html->link(__('List Tags'), ['action' => 'index'], ['class' => 'side-nav-item']) ?>
            <?= $this->Html->link(__('New Tag'), ['action' => 'add'], ['class' => 'side-nav-item']) ?>
        </div>
    </aside>
    <div class="column column-80">
        <div class="tags view content">
            <h3><?= h($tag->title) ?></h3>
            <table>
                <tr>
                    <th><?= __('Title') ?></th>
                    <td><?= h($tag->title) ?></td>
                </tr>
                <tr>
                    <th><?= __('Id') ?></th>
                    <td><?= $this->Number->format($tag->id) ?></td>
                </tr>
                <tr>
                    <th><?= __('Created') ?></th>
                    <td><?= h($tag->created) ?></td>
                </tr>
                <tr>
                    <th><?= __('Modified') ?></th>
                    <td><?= h($tag->modified) ?></td>
                </tr>
            </table>
```

```php
<div class="related">
    <h4><?= __('Related Articles') ?></h4>
    <?php if (!empty($tag->articles)) : ?>
    <div class="table-responsive">
        <table>
            <tr>
                <th><?= __('Id') ?></th>
                <th><?= __('Title') ?></th>
                <th><?= __('Content') ?></th>
                <th><?= __('Created') ?></th>
                <th><?= __('Modified') ?></th>
                <th><?= __('User Id') ?></th>
                <th class="actions">
                <?= __('Actions') ?></th>
            </tr>
            <?php foreach ($tag->articles as $article) : ?>
            <tr>
                <td><?= h($article->id) ?></td>
                <td><?= h($article->title) ?></td>
                <td><?= h($article->content) ?></td>
                <td><?= h($article->created) ?></td>
                <td><?= h($article->modified) ?></td>
                <td><?= h($article->user_id) ?></td>
                <td class="actions">
                    <?= $this->Html->link(__('View'), ['controller' => 'Articles', 'action' => 'view', $article->id]) ?>
                    <?= $this->Html->link(__('Edit'), ['controller' => 'Articles', 'action' => 'edit', $article->id]) ?>
                    <?= $this->Form->postLink(
                        __('Delete'),
                        ['controller' => 'Articles', 'action' => 'delete', $article->id],
                        [
                            'method' => 'delete',
                            'confirm' => __('Are you sure you want to delete # {0}?', $article->id),
                        ]
                    ) ?>
                </td>
            </tr>
            <?php endforeach; ?>
        </table>
    </div>
    <?php endif; ?>
</div>
```

```php
            <div class="related">
                <h4><?= __('Related Articles Tags') ?></h4>
                <?php if (!empty($tag->articles_tags)) : ?>
                <div class="table-responsive">
                    <table>
                        <tr>
                            <th><?= __('Article Id') ?></th>
                            <th><?= __('Tag Id') ?></th>
                            <th><?= __('Priority') ?></th>
                            <th class="actions"><?= __('Actions') ?></th>
                        </tr>
                        <?php foreach ($tag->articles_tags as $articlesTag) : ?>
                        <tr>
                            <td><?= h($articlesTag->article_id) ?></td>
                            <td><?= h($articlesTag->tag_id) ?></td>
                            <td><?= h($articlesTag->priority) ?></td>
                            <td class="actions">
                                <?= $this->Html->link(__('View'), ['controller' => 'ArticlessTags', 'action' => 'view', $articlesTag->article_id]) ?>
                                <?= $this->Html->link(__('Edit'), ['controller' => 'ArticlessTags', 'action' => 'edit', $articlesTag->article_id]) ?>
                                <?= $this->Form->postLink(
                                    __('Delete'),
                                    ['controller' => 'ArticlessTags', 'action' => 'delete', $articlesTag->article_id],
                                    [
                                        'method' => 'delete',
                                        'confirm' => __('Are you sure you want to delete # {0}?', $articlesTag->article_id),
                                    ]
                                ) ?>
                            </td>
                        </tr>
                        <?php endforeach; ?>
                    </table>
                </div>
                <?php endif; ?>
            </div>
        </div>
    </div>
</div>
```

Activer V
Accédez au

- **Le résultat :**



➢ Ajout d'une table entre Articles et Tags

Migrations

- – Création de la table articles_tags
- ➢ Génération de la table dans les fichiers de migrations

```
C:\wamp64\www\blog_app\bin>cake bake controller articles_tags
Baking controller class for ArticlesTags...

Creating file C:\wamp64\www\blog_app\src\Controller\ArticlesTagsController.php
Wrote `C:\wamp64\www\blog_app\src\Controller\ArticlesTagsController.php`
Bake is detecting possible fixtures...

Baking test case for App\Controller\ArticlesTagsController ...

Creating file C:\wamp64\www\blog_app\tests\TestCase\Controller\ArticlesTagsControllerTest.php
Wrote `C:\wamp64\www\blog_app\tests\TestCase\Controller\ArticlesTagsControllerTest.php`
Done
```

- ➢ Fichier de migrations CreateArticlesTags.

```php
    */
    public function up(): void
    {
        $this->execute('CREATE TABLE articless_tags (
            article_id INT NOT NULL,
            tag_id INT NOT NULL,
            priority INT NOT NULL CHECK (priority BETWEEN 1 AND 5),
            PRIMARY KEY (article_id, tag_id),
            FOREIGN KEY (tag_id) REFERENCES tags(id),
            FOREIGN KEY (article_id) REFERENCES articles(id)
        );'
    );
    }

    public function down(): void
    {
        $this->execute(
            'DROP table articles_tags;'
        );
    }
```

➢ Relation & Validateurs - ArticleTagsTable

➢ Création du fichier table de ArticleTagsTable

- Partie **initialisation** :

```
public function initialize(array $config): void
{
    parent::initialize($config);

    $this->setTable('articles_tags');
    $this->setDisplayField('article_id');
    $this->setPrimaryKey('id');

    $this->addBehavior('Timestamp');

    $this->belongsTo('Articles', [
        'foreignKey' => 'article_id',
        'joinType' => 'INNER',
    ]);
    $this->belongsTo('Tags', [
        'foreignKey' => 'tag_id',
        'joinType' => 'INNER',
    ]);
}
```

- Partie **Validateurs** :

```
public function validationDefault(Validator $validator): Validator
{
    $validator
        ->integer('priority')
        ->requirePresence('priority', 'create')
        ->notEmptyString('priority')
        ->add('priority', 'validValue', [
            'rule' => ['range', 1, 5]
            ]);


    $validator
        ->scalar('article_id')
        ->maxLength('article_id', 255)
        ->notEmptyString('article_id');

    $validator
        ->scalar('tag_id')
        ->maxLength('tag_id', 255)
        ->notEmptyString('tag_id');

    return $validator;
}
```

- Partie **bindRules** :

```
public function buildRules(RulesChecker $rules): RulesChecker
{
    $rules->add($rules->isUnique(['article_id', 'tag_id']), ['errorField' => 'article_id']);
    $rules->add($rules->existsIn(['article_id'], 'Articles'), ['errorField' => 'article_id']);
    $rules->add($rules->existsIn(['tag_id'], 'Tags'), ['errorField' => 'tag_id']);

    return $rules;
}
```

➢ ArticleTagsController

- Action index

```
public function index()
{
    $query = $this->ArticlesTags->find()
        ->contain(['Articles', 'Tags']);
    $articlesTags = $this->paginate($query);

    $this->set(compact('articlesTags'));
}
```

- Action view

```
 */
public function view($id = null)
{
    $articlesTag = $this->ArticlesTags->get($id, contain: ['Articles', 'Tags']);
    $this->set(compact('articlesTag'));
}
```

- Action add

```
 */
public function add()
{
    $articlesTag = $this->ArticlesTags->newEmptyEntity();
    if ($this->request->is('post')) {
        $articlesTag = $this->ArticlesTags->patchEntity($articlesTag, $this->request->getData());
        if ($this->ArticlesTags->save($articlesTag)) {
            $this->Flash->success(__('The articles tag has been saved.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The articles tag could not be saved. Please, try again.'));
    }
    $articles = $this->ArticlesTags->Articles->find('list', limit: 200)->all();
    $tags = $this->ArticlesTags->Tags->find('list', limit: 200)->all();
    $this->set(compact('articlesTag', 'articles', 'tags'));
}
```

- Action edit

```
 */
public function edit($id = null)
{
    $articlesTag = $this->ArticlesTags->get($id, contain: []);
    if ($this->request->is(['patch', 'post', 'put'])) {
        $articlesTag = $this->ArticlesTags->patchEntity($articlesTag, $this->request->getData());
        if ($this->ArticlesTags->save($articlesTag)) {
            $this->Flash->success(__('The articles tag has been saved.'));

            return $this->redirect(['action' => 'index']);
        }
        $this->Flash->error(__('The articles tag could not be saved. Please, try again.'));
    }
    $articles = $this->ArticlesTags->Articles->find('list', limit: 200)->all();
    $tags = $this->ArticlesTags->Tags->find('list', limit: 200)->all();
    $this->set(compact('articlesTag', 'articles', 'tags'));
}
```

- Action delete

```
 */
public function delete($id = null)
{
    $this->request->allowMethod(['post', 'delete']);
    $articlesTag = $this->ArticlesTags->get($id);
    if ($this->ArticlesTags->delete($articlesTag)) {
        $this->Flash->success(__('The articles tag has been deleted.'));
    } else {
        $this->Flash->error(__('The articles tag could not be deleted. Please, try again.'));
    }

    return $this->redirect(['action' => 'index']);
}
```

- Action deleteAT

Information à compléter du Contrôleur lorsque ça sera correct

```
public function deleteAT($idArt = null, $idTag = null) {
    //debug($idArt);
    //debug($idTag);
    //die;
    $this->request->allowMethod(['post', 'delete']);
    $articlesTag = $this->ArticlesTags->find()
            ->where([
                'article_id' => $idArt, 'tag_id' => $idTag])
            ->first();
    if ($this->ArticlesTags->delete($articlesTag)) {
        $this->Flash->success(__('The articles tag has been deleted.'));
    } else {
        $this->Flash->error(__('The articles tag could not be deleted. Please, try again.'));
    }

    return $this->redirect(['controller' => 'Articles', 'action' => 'detail', $idArt]);
}
```

Dans detail :

```
class="related">
        <h4><?= __('Tags Associés') ?></h4>
        <?php if (!empty($leArticle->tags)) : ?>
        <div class="table-responsive">
            <table>
                <tr>
                    <th><?= __('Id') ?></th>
                    <th><?= __('Title') ?></th>
                    <th><?= __('Modified') ?></th>
                    <th><?= __('Priority') ?></th>
                    <th class="actions"><?= __('Actions') ?></th>
                </tr>
                <?php foreach ($leArticle->tags as $tag) : ?>
                <tr>
                    <td><?= h($tag->ArticlesTags['tag_id']) ?></td>
                    <td><?= h($tag->title) ?></td>
                    <td><?= h($tag->modified->format(DATE_RFC850)) ?></td>
                    <td><?= h($tag->ArticlesTags['priority']
                            ) ?></td>
                    <td class="actions">
                        <?= $this->Html->link(__('View'), ['controller' => 'Tags', 'action' => 'view', $tag->ArticlesTags['tag_id']]) ?>
                        <?= $this->Html->link(__('Edit'), ['controller' => 'Articles', 'action' => 'edit', $tag->ArticlesTags['tag_id']]) ?>
                        <?= $this->Form->postLink(
                            __('Delete'),
                            ['controller' => 'ArticlesTags', 'action' => 'deleteAT', $leArticle->id, $tag->ArticlesTags['tag_id']],
                            [
                                'method' => 'delete',
                                'confirm' => __('Are you sure you want to delete # {0}?', $tag->ArticlesTags['tag_id']),
                            ]
                        ) ?>
                    </td>
                </tr>
                <?php endforeach; ?>
            </table>
        </div>
```

Activer Windows

The articles tag has been deleted.

1

1

Created : Friday, 19-Dec-25 03:51:53 UTC

Auteur : 1

Tags Associés

Id	Title	Modified	Priority	Actions
3	NSI	Friday, 19-Dec-25 03:51:54 UTC	3	View Edit Delete
4	Math Info	Friday, 19-Dec-25 03:51:54 UTC	3	View Edit Delete

Commentaires (0)

➢ Rework de ArticlesController

- Action index :

- ➢ Rajouter les tags dans le contain de mesArticles
- ➢ Rajouter orderBy pour pouvoir modifier par **ordre décroissant** des articles modifiés

```
public function index() {
    //on récupère tous les posts et on les stocke dans $mesArticles
    $mesArticles = $this->Articles->find()->contain([
                'Users' => function ($q) {
                    return $q
                            ->select(['username', 'email']);
                }, 'Comments' => function ($q) {
                    return $q
                            ->select(['article_id']);
                }, 'Tags' => function ($q) {
                    return $q
                            ->select(['ArticlesTags.article_id']);
                }])
            ->orderBy(['Articles.modified' => 'DESC'])
            ->all();

    $this->set(compact('mesArticles'));
}
```

- Action édit :

- ➢ Modifier mesTags (anciennement mesUsers) pour rajouter la liste des tags :

```
public function edit($id = null) {
    $mesTags = $this->fetchTable('Tags')
            ->find('list', keyField: 'id', valueField: 'title')
            ->toArray();
```

- ➢ Dans le try de l'action edit du leArticle : rajouter le contain tags en sélectionnant l'id de l'article_id, tag_id et la priorité de ArticlesTags.

```
            ->toArray();
    try {
        $leArticle = $this->Articles->get($id,
                contain: ['Tags' => function ($q) {
                        return $q
                                ->select(['ArticlesTags.article_id',
                                    'ArticlesTags.tag_id',
                                    'ArticlesTags.priority']);
                    }
        ]);
```

- ➢ Après la condition de requête « post » on modifie le code pour permettre la récupération des tags et priorité pour pouvoir les sélectionner dans le formulaire d'edit

```php
if ($this->request->is(['post', 'put'])) {

    $data = $this->request->getData();

    //newData contient les données à enregistrer dans la base (articles et articles_tags
    $newData['title'] = $data['title'];
    $newData['content'] = $data['content'];

    //Pour enregistrer les tags en meme temps, il faut la structure tags._joinData
    $newData['tags'] = [];

    // Nettoyer les données des tags
    if (!empty($data['tags']['_ids'])) {
        foreach ($data['tags']['_ids'] as $tagId) {
            //On crée la structure qui permetre d'enregistrer dans la table de jointure grace à _joinData
            foreach ($data['tags']['_joinData'] as $tag => $prio) {
                $tab = [];
                if ($tag == $tagId) {
                    $tab = ['id' => $tagId, '_joinData' => $prio];
                    $newData['tags'][] = $tab;
                }
            }
        }
    }

    $this->Articles->patchEntity($leArticle, $newData);
    $leArticle->user_id = $this->request->getAttribute('identity')->id;
    if ($this->Articles->save($leArticle,
                    ['associated' => ['Tags._joinData']])) {
```

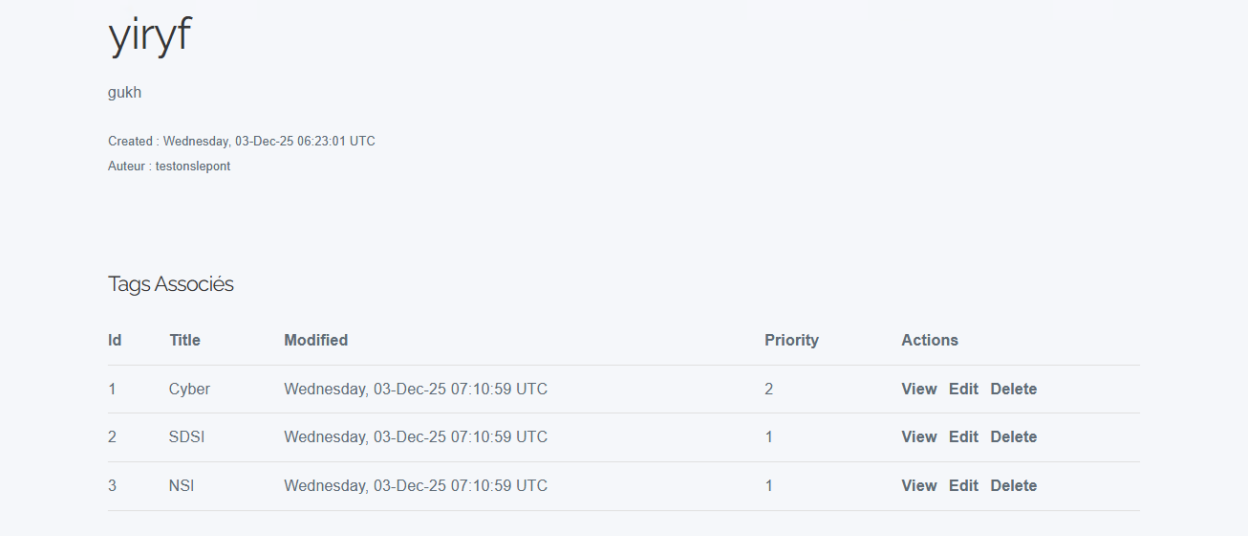
Le résultat attendu :

- Action détail :

- Rajouter dans le contain de leArticle la table Tags pour pouvoir afficher en détail sur un article le tag de l'id, son titre, sa date de modification et sa priorité

```
try {
    $leArticle = $this->Articles->get($id,
            contain: ['Comments.Users' => function ($q) {
                    return $q
                            ->select('Users.username')
                            ->order(['Comments.created asc']);
                },
                'Users' => function ($q) {
                    return $q
                            ->select(['Users.username']);
                },
                'Tags' => function ($q) {
                    return $q
                            ->select(['ArticlesTags.tag_id',
                                'Tags.title',
                                'Tags.modified',
                                'ArticlesTags.priority']);

                }]);
```

Le résultat attendu dans « Tags Associés » :



- Action add

- Après la condition de requête « post » on modifie le code pour permettre la récupération des tags et priorité pour pouvoir les sélectionner dans le formulaire d'ajout

```php
$mesTags = $this->fetchTable('Tags')
        ->find('list', keyField: 'id', valueField: 'title')
        ->toArray();
$leNewArticle = $this->Articles->newEmptyEntity();

if ($this->request->is('post')) {

    //on recupere le data du formulaire pour recreer un tableau tag/priorite
    $data = $this->request->getData();

    //newData contient les données à enregistrer dans la base (articles et articles_tags
    $newData['title'] = $data['title'];
    $newData['content'] = $data['content'];

    //Pour enregistrer les tags en meme temps, il faut la structure tags._joinData
    $newData['tags'] = [];

    // Nettoyer les données des tags
    if (!empty($data['tags']['_ids'])) {
        foreach ($data['tags']['_ids'] as $tagId) {
            //On crée la structure qui permetre d'enregistrer dans la table de jointure grace à _joinData
            foreach ($data['tags']['_joinData'] as $tag => $prio) {
                $tab = [];
                if ($tag == $tagId) {
                    $tab = ['id' => $tagId, '_joinData' => $prio];
                    $newData['tags'][] = $tab;
                }
            }
        }
    }

    //on recupere l'id du user connecté et on le rajoute en tant que clé étrangere
    $leNewArticle->user_id = $this->request->getAttribute('identity')->id;

    $leNewArticle = $this->Articles->patchEntity($leNewArticle, $newData,
            ['associated' => ['Tags._joinData']]);
```

Activer Windows

Le résultat attendu :

➢ Comptage

- Nombre d'articles sur les tags

➢ Ajouter la colonne « Nb Articles » dans une balise <th>, il n'y a pas besoin de pagination car le triage ne se fera pas

```
<thead>
    <tr>
        <th><?= $this->Paginator->sort('id') ?></th>
        <th><?= $this->Paginator->sort('title') ?></th>
        <th> Nb Articles </th>
        <th><?= $this->Paginator->sort('created') ?></th>
        <th><?= $this->Paginator->sort('modified') ?></th>
        <th class="actions"><?= __('Actions') ?></th>
    </tr>
</thead>
```

➢ Dans la vue index de Tags, rajouter la fonction count() dans un <td>

```
tbody>
  <?php foreach ($tags as $tag): ?>
  <tr>
      <td><?= $this->Number->format($tag->id) ?></td>
      <td><?= h($tag->title) ?></td>
      <td><?= count($tag->articles) ?></td>
      <td><?= h($tag->created) ?></td>
      <td><?= h($tag->modified) ?></td>
      <td class="actions">
          <?= $this->Html->link(__('View'), ['action' => 'view',
```

- Nombre tags sur les articles

➢ Rajouter la colonne « Nb Tags »

```
<table>
    <tr>
        <th>Id</th>
        <th>Titre</th>
        <th>Créé par</th>
        <th>Nb commentaires</th>
        <th>Nb tags</th>
        <th>Date de création</th>
        <th>Date de modification</th>
        <?php
            $identity = $this->request->getAtt
            if ($identity) {
        ?>
```

Le résultat attendu :

Tags

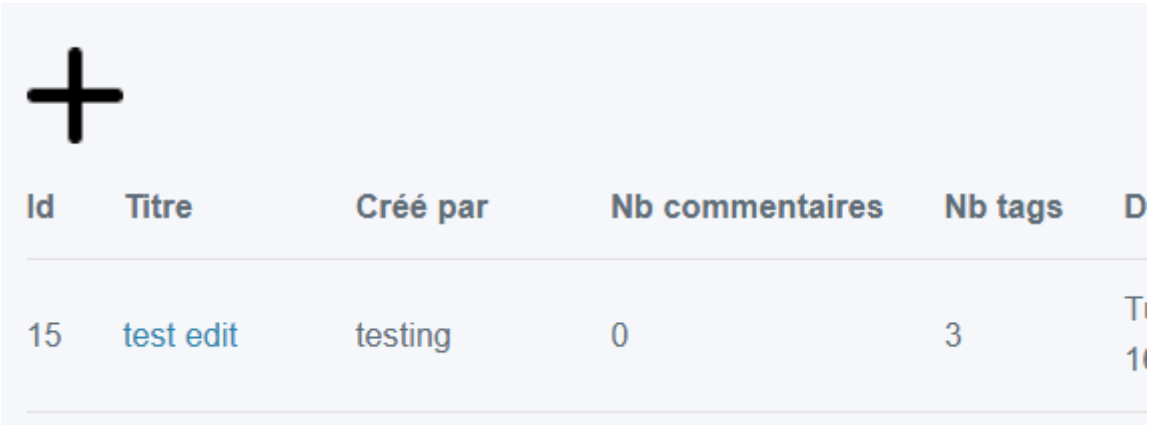
NEW TAG

Id	Title	Nb Articles	Created	Modified	Actions
1	Cyber	4	11/26/25, 11:44 AM	12/9/25, 4:15 PM	View Edit Delete
2	SDSI	1	11/26/25, 11:48 AM	12/3/25, 7:10 AM	View Edit Delete
3	NSI	3	11/26/25, 11:48 AM	12/9/25, 4:15 PM	View Edit Delete
4	Math Info	2	11/26/25, 11:50 AM	12/9/25, 4:15 PM	View Edit Delete
6	test	0	11/28/25, 3:41 AM	11/28/25, 3:41 AM	View Edit Delete

- Dans la vue index de Articles, rajouter la fonction count()

```
<td><?= $article->id ?></td>
<td>
    <?=
    $this->html->link($article->title, [
        'controller' => 'articles',
        'action' => 'detail',
        $article->id]);
    //l'url généré sera de la forme /articles/detail/…
    ?>
</td>

<td><?= $article->user->username ?></td>
<td><?= count($article->comments) ?></td>
<td><?= count($article->tags) ?></td>
<td><?= $article->created->format(DATE_RFC850) ?></td>
<td><?= $article->modified->format(DATE_RFC850) ?></td>


<td> <?php $identity = $this->request->getAttribute('identity');
if ($identity && ($article->user_id == $identity->id)) {
echo
$this->html->link($this->html->image("btn_edit.png", ["alt" => "edit"]),
            ['action' => 'edit', $article->id,
            'style' => 'height:48px;'],
            ['escape' => false]);
```

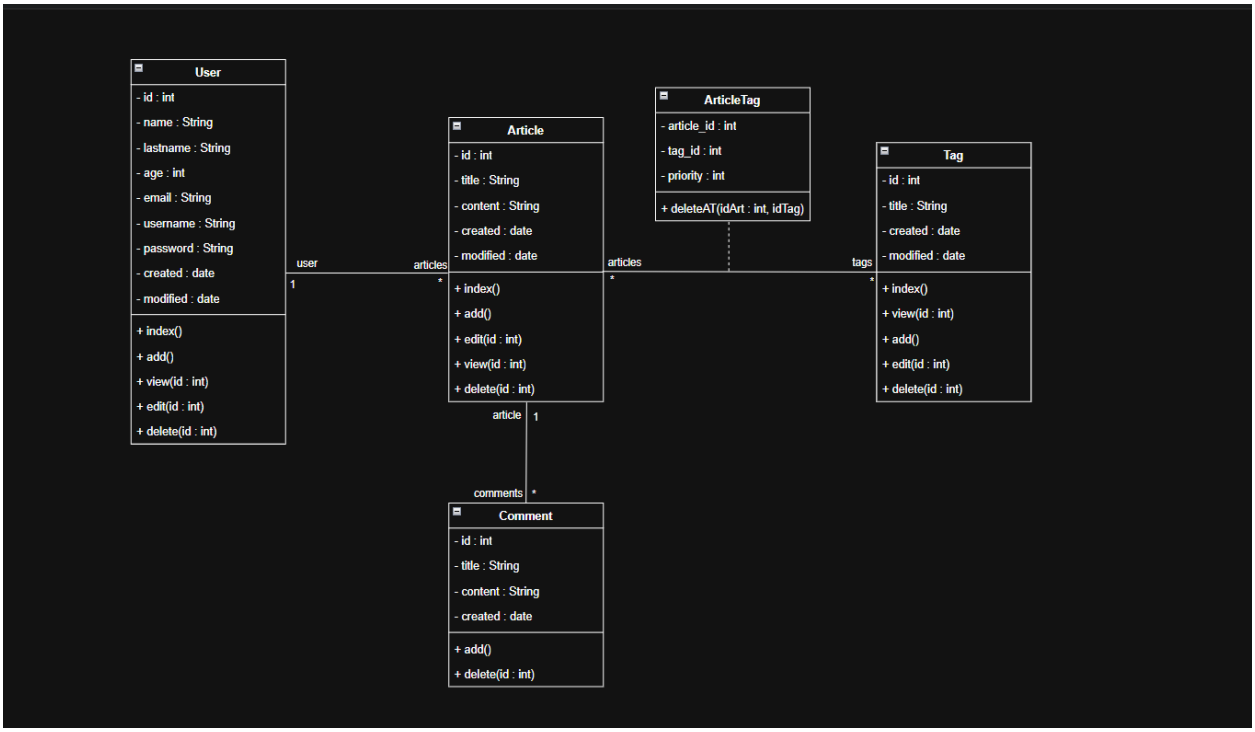
Le résultat attendu :



Id	Titre	Créé par	Nb commentaires	Nb tags	D
15	test edit	testing	0	3	T 1

Diagramme de classe final

Le diagramme final du projet ci-dessous :



RECAP' DES ÉTAPES DE BASE

- ➔ Créer un fichier de migration (migrate/rollback)
- ➔ Créer la table dans modèle (fichier avec la terminaison Table.php)
  - Ajouter les relations (hasMany, belongsTo, hasOne, belongsToMany si besoin)
  - Créer les validateurs
- ➔ Créer le contrôleur (fichier avec la terminaison Controller.php)
  - Créer les actions/fonctions dans le contrôleur (ex : index, add, edit, delete, view)
- ➔ Créer les vues dans src/templates
  - Dossier avec le nom de la classe
  - Fichiers des actions du contrôleur